New L J Institute of Engineering and Technology, Bodakdev

Branch: IT

Semester: 6

Subject: Software Engineering
IMP Question

1) Explain Software Engineering as a Layered Technology.

Software engineering is a fully layered technology, to develop software we need to go from one layer to another. All the layers are connected and each layer demands the fulfillment of the previous layer.

Fig: The diagram shows the layers of software development

Layered technology is divided into four parts:

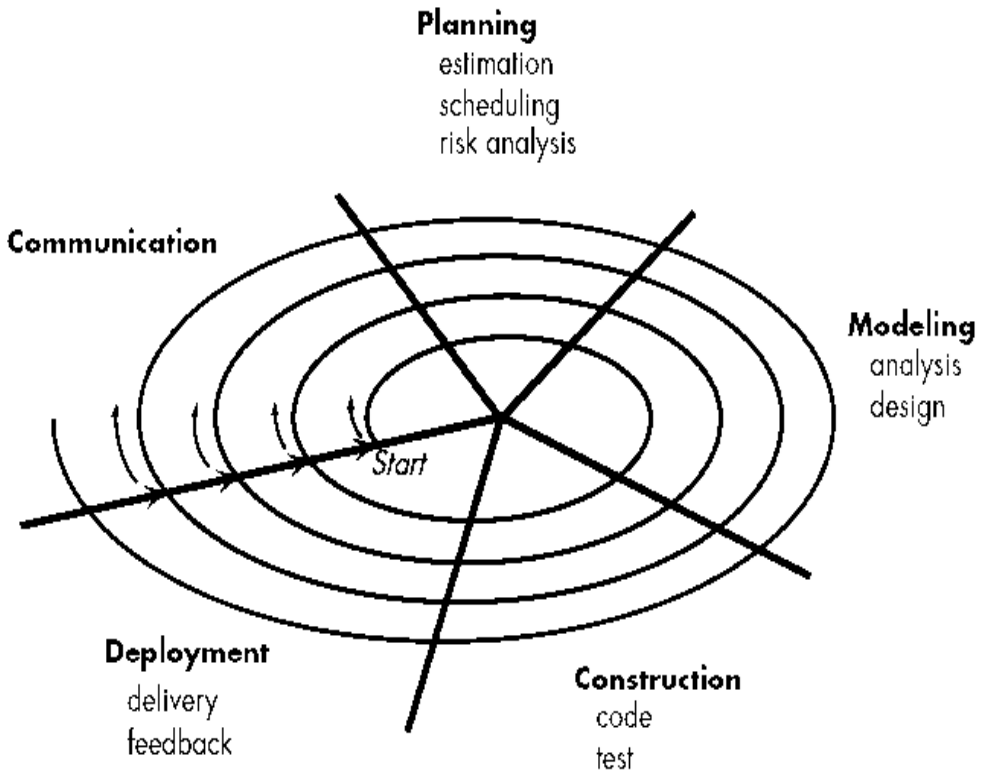
1. **A quality focus**: It defines the continuous process improvement principles of software. It provides integrity that means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data. It also focuses on maintainability and usability.

2. Process: It is the foundation or base layer of software engineering. It is key that binds all the layers together which enables the development of software before the deadline or on time. Process defines a framework that must be established for the effective delivery of software engineering technology. The software process covers all the activities, actions, and tasks required to be carried out for software development.

3. Method: During the process of software development the answers to all "how-to-do" questions are given by method. It has the information of all the tasks which includes communication, requirement analysis, design modeling, program construction, testing, and support.

4. Tools: Software engineering tools provide a self-operating system for processes and methods. Tools are integrated which means information created by one tool can be used by another.

2) Explain Spiral Model in Detail. State its Advantages and Disadvantages.



It provides the potential for rapid development.
• Software is developed in a series of evolutionary releases.
• Early iteration release might be prototype but later iterations provides more complete version of software.
• It is divided into framework activities. Each activity represent one segment of the spiral
• Each pass through the planning region results in adjustments to
♣ the project plan ♣ Cost & schedule based on feedback

When to use Spiral Model?

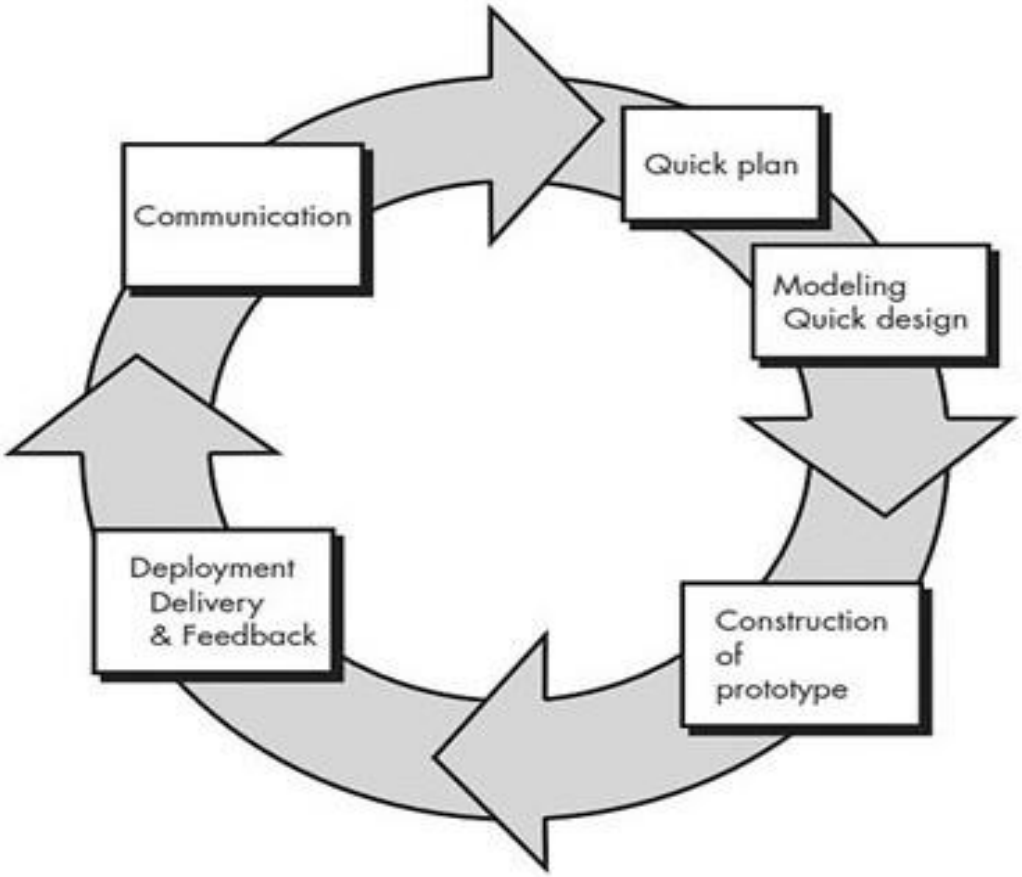
- For development of large scale / high-risk projects.
- When costs and risk evaluation is important.
- Users are unsure of their needs.
- Requirements are complex.
- New product line.
- Significant (considerable) changes are expected.

Advantages
• High amount of risk analysis hence, avoidance of Risk is enhanced.
• Strong approval and documentation control.
• Additional functionality can be added at a later date.
• Software is produced early in the Software Life Cycle.

Disadvantages

• Can be a costly model to use.
• Risk analysis requires highly specific expertise.
• Project's success is highly dependent on the risk analysis phase.
• Doesn't work well for smaller projects.

3) Explain steps involved during the prototyping.



- Communicate with stackholders & define objective of Software
- Identify requirements & design quick plan
- Model a quick design (focuses on visible part of software)
- Construct Prototype & deploy
- Stakeholders evaluate this prototype and provides feedback
- Iteration occurs and prototype is tuned based on feedback

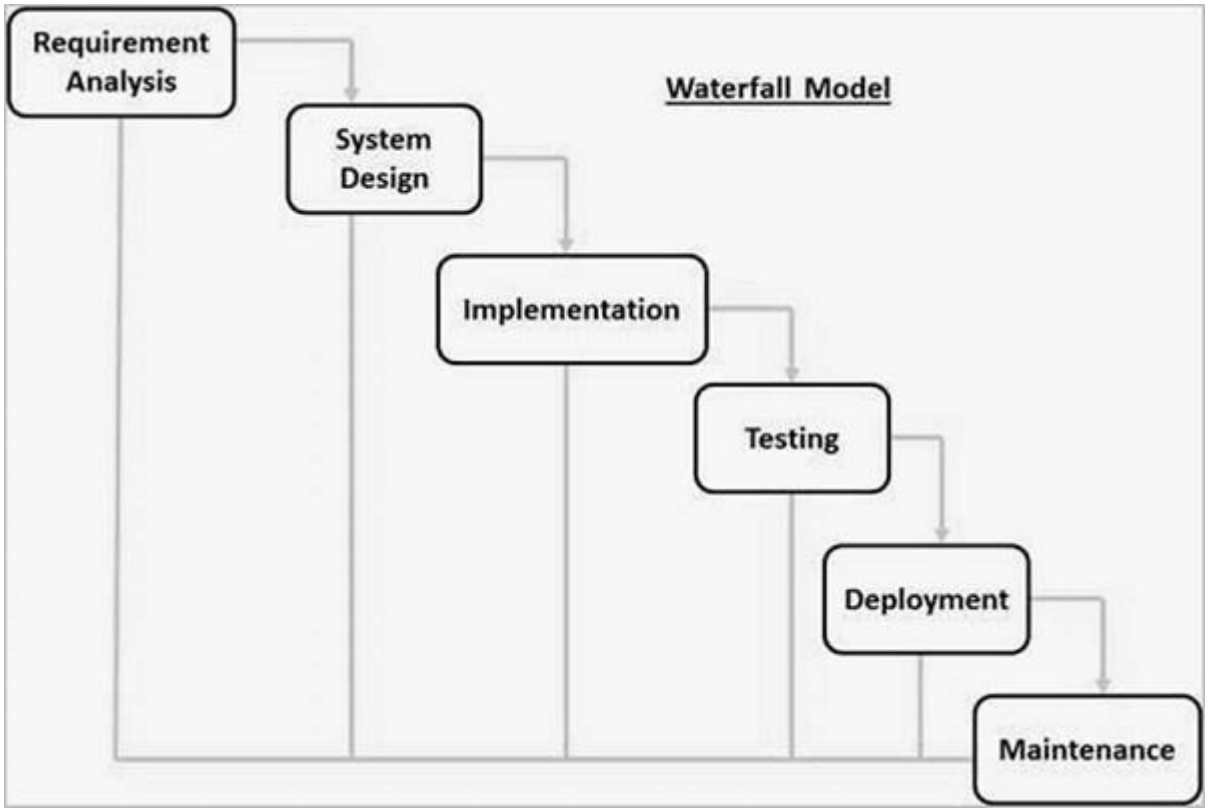
Advantage of Prototype Model

1. Reduce the risk of incorrect user requirement
2. Good where requirement are changing/uncommitted
3. Regular visible process aids management
4. Support early product marketing
5. Reduce Maintenance cost.
6. Errors can be detected much earlier as the system is made side by side.

Disadvantage of Prototype Model

1. An unstable/badly implemented prototype often becomes the final product.
2. Difficult to know how long the project will last.
3. Easy to fall back into the code and fix without proper requirement analysis, design, customer evaluation, and feedback.
4. Prototyping tools are expensive.
5. Special tools & techniques are required to build a prototype.
6. It is a time-consuming process.

4) Explain Waterfall Model with suitable diagram.



The sequential phases in Waterfall model are −

- **Requirement Gathering and analysis** − All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- **System Design** − The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- **Implementation** − With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.
- **Integration and Testing** − All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.

- **Deployment of system** − Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
- **Maintenance** − There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

Advantages of Waterfall model

- This model is simple to implement also the number of resources that are required for it is minimal.
- The requirements are simple and explicitly declared; they remain unchanged during the entire project development.
- The start and end points for each phase is fixed, which makes it easy to cover progress.
- The release date for the complete product, as well as its final cost, can be determined before development.
- It gives easy to control and clarity for the customer due to a strict reporting system.

Disadvantages of Waterfall model

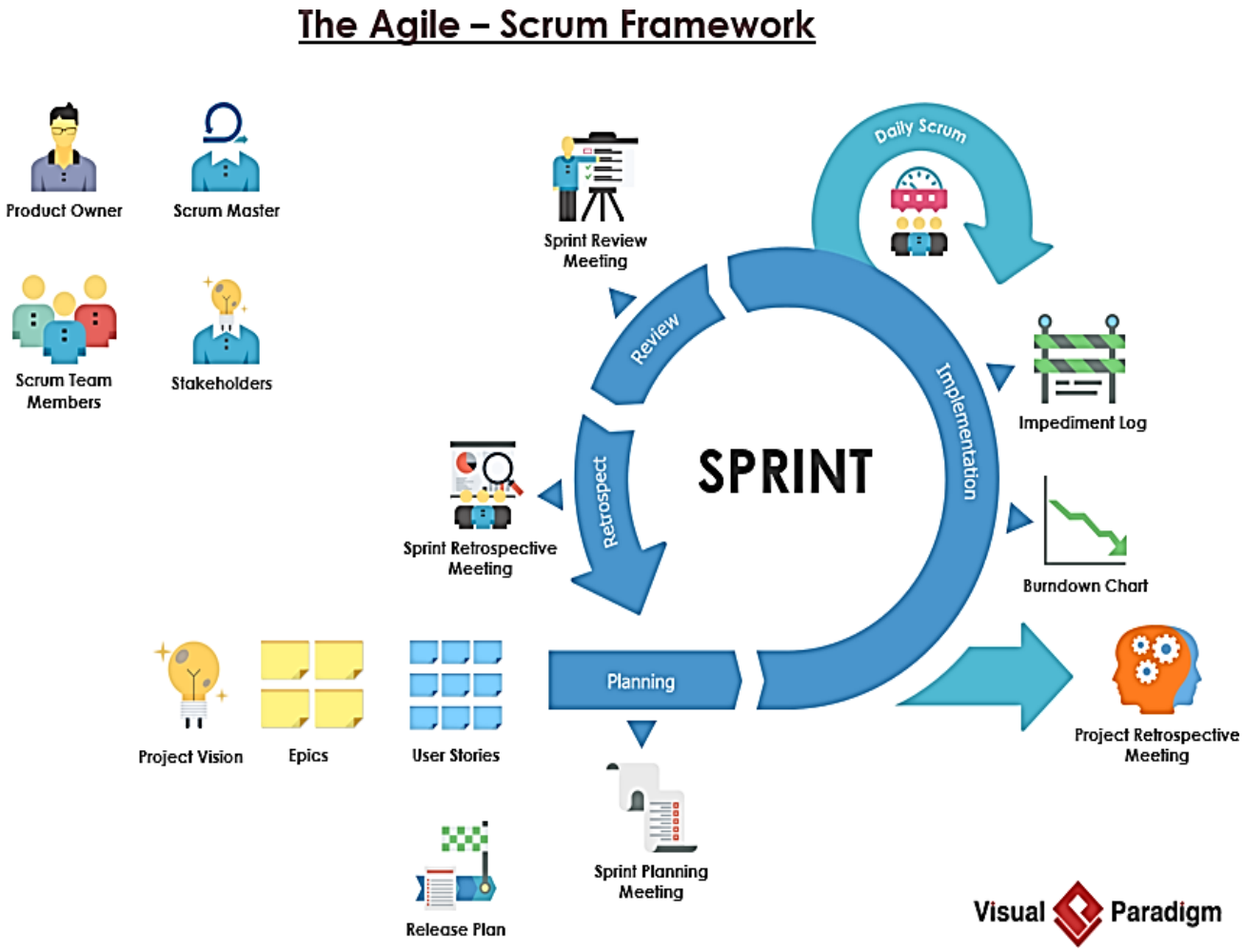
- In this model, the risk factor is higher, so this model is not suitable for more significant and complex projects.
- This model cannot accept the changes in requirements during development.
- It becomes tough to go back to the phase. For example, if the application has now shifted to the coding phase, and there is a change in requirement, It becomes tough to go back and change it.
- Since the testing done at a later stage, it does not allow identifying the challenges and risks in the earlier phase, so the risk reduction strategy is difficult to prepare.

5) Compare Prototype and RAD Process Model.

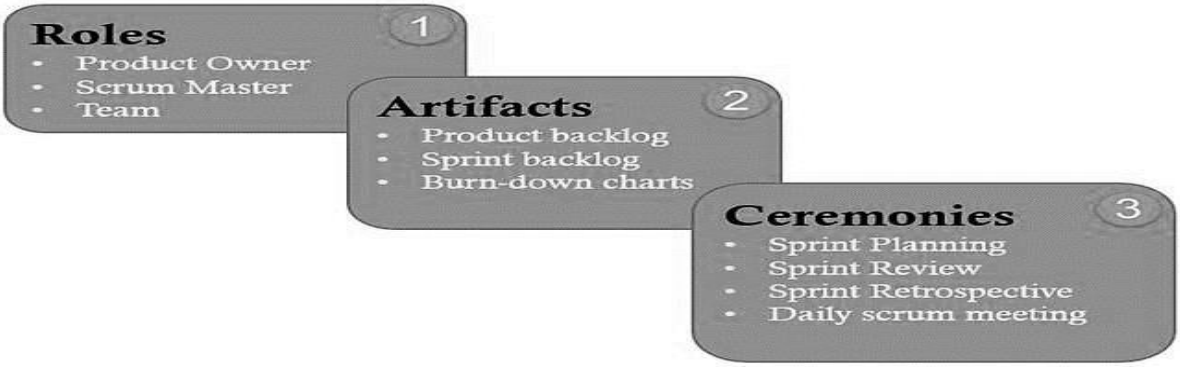
S.No.	Prototype Model	RAD Model
1.	A prototype model is a software development model where a prototype is built, tested and then refined as per customer needs.	RAD model is a software development model where by the components or functions are developed in parallel as if they were mini projects.
2.	Phases- 1. Requirement Analysis 2. Quick Design 3. Build Prototype 4. User Evaluation 5. Refining Prototype 6. Implement and Maintain	Phases- 1. Business Modelling 2. Data Modelling 3. Process Modelling 4. Application Generation 5. Testing and Turnover
3.	It is suitable for high-risk projects.	It is not suitable when technical risks are high.
4.	It does not give emphasis on risk analysis.	It give emphasis on risk analysis.

5.	In this, quick initial reviews are not possible.	In this, quick initial reviews are possible.
6.	It does not support automatic code generation.	It supports automatic code generation as. results in minimal code writing.
7.	It is best suited when the requirement of the client is not clear and supposed to be changed.	It is best suited when the customer requirements are clear.
8.	In this user involvement is high.	In this user Involvement is only at the beginning.
9.	Low team size is sufficient.	It requires a large team to carry out the operations.
10.	End-Product cannot be developed in less time.	End-Product can be developed in less time.

6) Explain Scrum with its advantages and disadvantages.



Scrum is based on the following 3 Pillars-



Lifecycle of Scrum:
Scrum is the type of Agile framework. It is a framework within which people can address complex adaptive problem while productivity and creativity of delivering product is at highest possible values. Scrum uses Iterative process. Silent features of Scrum are:

- Scrum is light-weighted framework
- Scrum emphasizes self-organization
- Scrum is simple to understand
- Scrum framework help the team to work together

Sprint: A Sprint is a time box of one month or less. A new Sprint starts immediately after the completion of the previous Sprint.
Release: When the product is completed, it goes to the Release stage.
Sprint Review: If the product still has some non-achievable features, it will be checked in this stage and then passed to the Sprint Retrospective stage.
Sprint Retrospective: In this stage quality or status of the product is checked.
Product Backlog: According to the prioritize features the product is organized.
Sprint Backlog: Sprint Backlog is divided into two parts Product assigned features to sprint and Sprint planning meeting.

Advantage of using Scrum framework:

- Scrum framework is fast moving and money efficient.
- Scrum framework works by dividing the large product into small sub-products. It's like a divide and conquer strategy
- In Scrum customer satisfaction is very important.
- Scrum is adaptive in nature because it have short sprint.
- As Scrum framework rely on constant feedback therefore the quality of product increases in less amount of time

Disadvantage of using Scrum framework:

- Scrum framework do not allow changes into their sprint.
- Scrum framework is not fully described model. If you wanna adopt it you need to fill in the framework with your own details like Extreme Programming(XP), Kanban, DSDM.
- It can be difficult for the Scrum to plan, structure and organize a project that lacks a clear definition.
- The daily Scrum meetings and frequent reviews require substantial resources.

7) Explain Extreme Programming (XP) in detail.

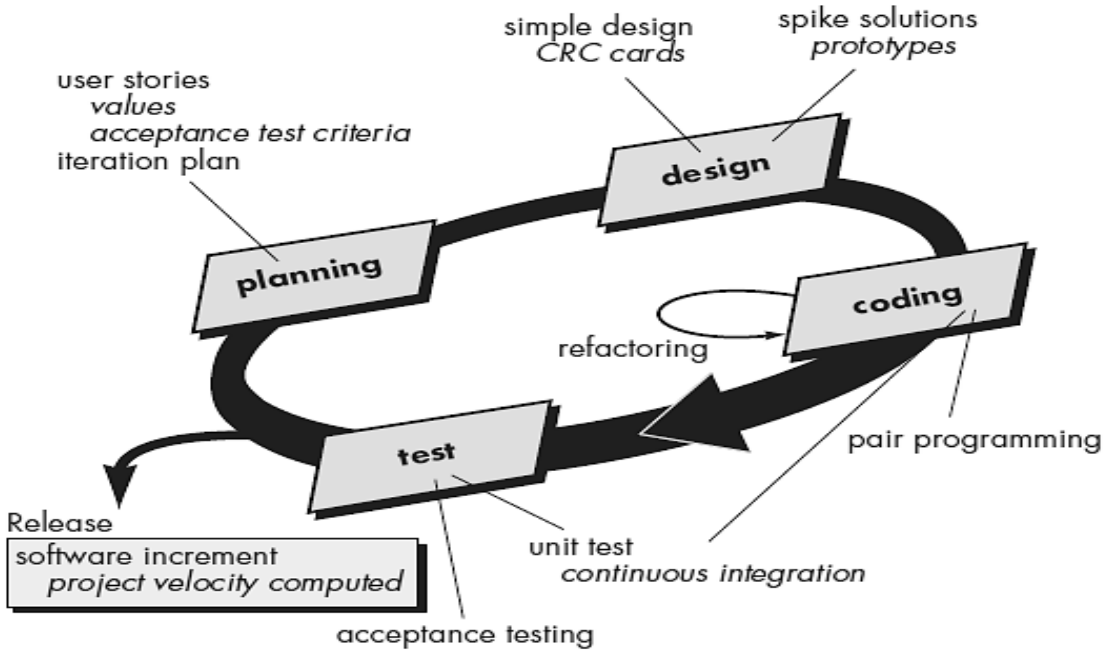
Extreme programming (XP) is one of the most important software development frameworks of Agile models. It is used to improve software quality and responsiveness to customer requirements. The extreme programming model recommends taking the best practices that have worked well in the past in program development projects to extreme levels.

- Originally described by Kent Beck, has emerged as one of the most popular and controversial Agile models.
- Lightweight, efficient, low-risk, flexible, predictable, scientific, and fun way to develop a software.
- eXtreme Programming (XP) conceived and developed to address the specific needs of software development by small teams in the face of ambiguous and changing requirements.
- Disciplined approach for high-quality agile software development, focused on speed and continuous delivery.

The XP Process :

It considers 4 framework activities

- Planning
- Design
- Coding
- Testing



<u>Planning :</u>

User Stories

- Customers assigns value (priority)
- Developers assigns cost (number of development weeks)
  Project velocity
- Computed at the end of first release

- Number of stories implemented in first release
- Estimates for future release
- Guard against over-commitment

Design:

- Keep-it-Simple (Design of extra functionality is discouraged)
- Preparation of CRC card is work project
    - CRC cards identify and organize object oriented classes
- Spike Solutions (in case of difficult design problem is encountered)
    - Operational prototype intended to clear confusion
- Refactoring
- Modify internals of code, No observable change

Class Name	
Responsibilities	Collaborators

Coding :

- Develops a series of Unit test for stories included in current release
- Complete code perform unit-test to get immediate feedback
- XP recommend pair-programming, "Two heads are better than one"
- Integrate code with other team members, this "continuous integration" helps to avoid compatibility & interfacing problems, "smoke testing" environment to uncover errors Early

Testing :

- Unit test by developers & fix small problems
- Acceptance tests - Specified by customer
- This encourages regression testing strategy whenever code is modified

XP Values :

Communication: To achieve effective communication, it emphasized close & informal (verbal) collaboration between customers and developers
Simplicity: It restricts developers to design for immediate needs not for future needs
Feedback: It is derived from three sources the implemented software, the customer and other software team members, it uses Unit testing as primary testing
Courage: It demands courage (discipline), there is often significant pressure to design for future requirements, XP team must have the discipline (courage) to design for today
Respect: XP team respect among members

8) Explain Agile Development in detail.

Agile Software Development is a software development methodology that values flexibility, collaboration, and customer satisfaction. It is based on the Agile Manifesto, a set of principles for software development that prioritize individuals and interactions, working software, customer collaboration, and responding to change.

Agile Software Development is an iterative and incremental approach to software development that emphasizes the importance of delivering a working product quickly and frequently. It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.

The Agile Software Development process typically consists of the following steps:

Requirements Gathering: The customer's requirements for the software are gathered and prioritized.
Planning: The development team creates a plan for delivering the software, including the features that will be delivered in each iteration.
Development: The development team works to build the software, using frequent and rapid iterations.
Testing: The software is thoroughly tested to ensure that it meets the customer's requirements and is of high quality.
Deployment: The software is deployed and put into use.
Maintenance: The software is maintained to ensure that it continues to meet the customer's needs and expectations.

Agile Software Development is widely used by software development teams and is considered to be a flexible and adaptable approach to software development that is well-suited to changing requirements and the fast pace of software development.

Agile is a time-bound, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver all at once.

Agile Testing Methods:
1. Scrum
2. Crystal
3. Dynamic Software Development Method(DSDM)
4. Feature Driven Development(FDD)
5. Lean Software Development
6. eXtreme Programming(XP)

9) Explain W5HH principle with respect to software project management.

Boehm proposes an approach that focuses on project objectives, milestones and timetables, responsibilities, management and technical approaches, and resources necessary. He calls it the principle WWWWWHH for a series of questions that lead to a definition of the main characteristics of the project and the resulting project plan:

1> Why is the system developed?
The answer to this question all parties to the validity of the business reasons for the software to evaluate. Set in a different way, time not only business people and spending money?

2>What to do until then?
Answers to these questions to help the team a project schedule to establish the identification of key project tasks and objectives that the client is required.

3>Who is responsible for a function?
Earlier in this chapter, we noted that the role and responsibility of each team member should be designated programs. The answer to this question helps achieve.

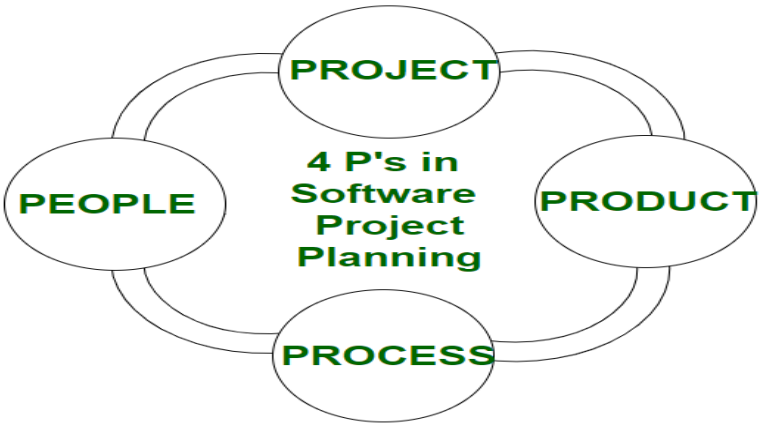
4> Where are they located organize?
Not all the roles and responsibilities of living in the software team itself. , Customer users and other stakeholders also have responsibilities.

5> How will the work done technically and managerially?
When product scope is established, a management and technical strategy for the project specified. How much of each resource is needed? The answer to this question is taking conducting evaluations based on answers to previous questions.

10) Define FOUR Ps for project management and explain them in detail.

For properly building a product, there's a very important concept that we all should know in software project planning while developing a product. There are 4 critical components in software project planning which are known as the 4P's namely:

1. Product
2. Process
3. People
4. Project



These components play a very important role in your project that can help your team meet its goals and objective. Now, Let's dive into each of them a little in detail to get a better understanding:

People

The most important component of a product and its successful implementation is human resources. In building a proper product, a well-managed team with clear-cut roles defined for each person/team will lead to the success of the product. We need to have a good team in order to save our time, cost, and effort. Some assigned roles in software project planning are project manager, team leaders, stakeholders, analysts, and other IT professionals. Managing people successfully is a tricky process which a good project manager can do.

Product

As the name inferred, this is the deliverable or the result of the project. The project manager should clearly define the product scope to ensure a successful result, control the team members, as well technical hurdles that he or she may encounter during the building of a product. The product can consist of both tangible or intangible such as shifting the company to a new place or getting a new software in a company.

Process

In every planning, a clearly defined process is the key to the success of any product. It regulates how the team will go about its development in the respective time period. The Process has several steps involved like, documentation phase, implementation phase, deployment phase, and interaction phase.

Project

The last and final P in software project planning is Project. It can also be considered as a blueprint of process. In this phase, the project manager plays a critical role. They are responsible to guide the team members to achieve the project's target and objectives, helping & assisting them with issues, checking on cost and budget, and making sure that the project stays on track with the given deadlines.

11) Explain COCOMO model for project estimation.

Boehm proposed COCOMO (Constructive Cost Estimation Model) in 1981.COCOMO is one of the most generally used software estimation models in the world. COCOMO predicts the efforts and schedule of a software product based on the size of the software.

The necessary steps in this model are:

Get an initial estimate of the development effort from evaluation of thousands of delivered lines of source code (KDLOC).

Determine a set of 15 multiplying factors from various attributes of the project.

Calculate the effort estimate by multiplying the initial estimate with all the multiplying factors i.e., multiply the values in step1 and step2.

The initial estimate (also called nominal estimate) is determined by an equation of the form used in the static single variable models, using KDLOC as the measure of the size. To determine the initial effort Ei in person-months the equation used is of the

type is shown below

Ei=a*(KDLOC)b

The value of the constant a and b are depends on the project type.

In COCOMO, projects are categorized into three types:

1. Organic
2. Semidetached
3. Embedded

1.Organic: A development project can be treated of the organic type, if the project deals with developing a well-understood application program, the size of the development team is reasonably small, and the team members are experienced in developing similar methods of projects. Examples of this type of projects are simple business systems, simple inventory management systems, and data processing systems.

2. Semidetached: A development project can be treated with semidetached type if the development consists of a mixture of experienced and inexperienced staff. Team members may have finite experience in related systems but may be unfamiliar with some aspects of the order being developed. Example of Semidetached system includes developing a new operating system (OS), a Database Management System (DBMS), and complex inventory management system.

3. Embedded: A development project is treated to be of an embedded type, if the software being developed is strongly coupled to complex hardware, or if the stringent regulations on the operational method exist. For Example: ATM, Air Traffic control.

For three product categories, Bohem provides a different set of expression to predict effort (in a unit of person month)and development time from the size of estimation in KLOC(Kilo Line of code) efforts estimation takes into account the productivity loss due to holidays, weekly off, coffee breaks, etc.

According to Boehm, software cost estimation should be done through three stages:

1. Basic Model
2. Intermediate Model
3. Detailed Model

1. Basic COCOMO Model: The basic COCOMO model provide an accurate size of the project parameters. The following expressions give the basic COCOMO estimation model:

E= a(KLOC)^b

time= c(Effort)^d

Person required = Effort/ time

Effort=a1*(KLOC) a2 PM

Tdev=b1*(efforts)b2 Months

Where

KLOC is the estimated size of the software product indicate in Kilo Lines of Code,

a1,a2,b1,b2 are constants for each group of software products,

Tdev is the estimated time to develop the software, expressed in months,

Effort is the total effort required to develop the software product, expressed in person months (PMs).

2. Intermediate Model: The basic Cocomo model considers that the effort is only a function of the number of lines of code and some constants calculated according to the various software systems. The intermediate COCOMO model recognizes these facts and refines the initial estimates obtained through the basic COCOMO model by using a set of 15 cost drivers based on various attributes of software engineering.

Software Projects	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

3. Detailed COCOMO Model: Detailed COCOMO incorporates all qualities of the standard version with an assessment of the cost driver?s effect on each method of the software engineering process. The detailed model uses various effort multipliers for each cost driver property. In detailed cocomo, the whole software is differentiated into multiple modules, and then we apply COCOMO in various modules to estimate effort and then sum the effort.

12) Explain Risk Management, Monitoring and Mitigation.

Following are three important issues or steps that must be considered for developing effective strategies:

- **Risk mitigation** (proactive planning for risk avoidance)
- **Risk monitoring** (assessing whether predicted risks occur or not, ensuring risk aversion steps are being properly applied, collect information for future risk analysis, attempt to determine which risks caused which problems)
- **Risk management** and contingency planning (actions to be taken in the event that mitigation steps have failed and the risk has become a live problem)

✓ The goal - identify as many potential risks as possible.

- ✓ When all risks have been identified, they will then be evaluated to determine their probability of occurrence, Plans will then be made to avoid each risk, to track each risk to determine if it is more or less likely to occur, and to plan for those risks should they occur.
- ✓ It is the organization's responsibility to perform risk mitigation, monitoring, and management in order to produce a quality product.
- ✓ The quicker the risks can be identified and avoided, the smaller the chances of having to face that particular risk's consequence.
- ✓ The fewer consequences suffered as a result of good **RMMM plan**, the better the product, and the smoother the development process.

Risk Mitigation :

- Risk mitigation (avoidance) is the primary strategy and is achieved through a plan
- For Ex., Risk of high staff turnover
    - To mitigate this risk, you would develop a strategy for reducing turnover.
    - The possible steps to be taken are:
        - Meet with current staff to determine causes for turnover (e.g., poor working conditions, low pay, and competitive job market).
        - Mitigate those causes that are under your control before the project starts.
        - Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave.
        - Organize project teams so that information about each development activity is widely dispersed.
        - Define work product standards and establish mechanisms to be sure that all models and documents are developed in a timely manner.
        - Conduct peer reviews of all work (so that more than one person is "up to speed").
        - Assign a backup staff member for every critical technologist.

Risk Monitoring :

- The project manager monitors factors that may provide an indication of whether the risk is becoming more or less likely.
- In the case of high staff turnover, the general attitude of team members based on project pressures, the degree to which the team has jelled, inter-personal relationships among team members, potential problems with compensation and benefits, and the availability of jobs within the company and outside it are all monitored.
- In addition to monitoring these factors, a project manager should monitor the effectiveness of risk mitigation steps.
- The project manager should monitor work products carefully to ensure that each can stand on its own and that each imparts information that would be necessary if a newcomer were forced to join the software team somewhere in the middle of the project.

Risk Management :

- Risk management and contingency planning assumes that mitigation efforts have failed and that the risk has become a reality.
- If the mitigation strategy has been followed, backup is available, information is

documented, and knowledge has been dispersed across the team.

- In addition, you can temporarily refocus resources (and readjust the project schedule) to those functions that are fully staffed, enabling newcomers who must be added to the team to "get up to speed." Those individuals who are leaving are asked to stop all work and spend their last weeks in "knowledge transfer mode."

13) Explain type of Software risk.

What is Risk ??

- Risk is an expectation of loss, a potential problem that may or may not occur in the future.
- A possibility of suffering from loss in software development process is called a software risk

- A risk is a potential problem – it might happens & it might not
- Conceptual definition of risk
    - Risk concerns future happenings
    - Risk involves change in mind, opinion, actions, places, etc.
    - Risk involves choice & the uncertainty that choice requires.
- Two characteristics of risk
    - Uncertainty – the risk may or may not happen, that is, there are no 100% risks (those, instead, are called constraints)
    - Loss – the risk becomes a reality & unwanted consequences or losses occur

Risk Categorization :

- Project risks
    - They threaten the project plan
    - If project risk become real, it is likely that the project schedule will slip & that costs will increase
    - Identifies problems related to budgetary, schedule, personnel & resource
- Technical risks
    - They threaten the quality & timeliness of the software to be produced
    - If they become real, implementation may become difficult or impossible
    - Identifies problems related to design, implementation, maintenance etc.
- Business risks
    - They threaten the viability of the software to be built
    - If they become real, they risk the project or the product
- **Sub-categories of Business risks**
    - Market risk – building an excellent product or system that no one really wants
    - Strategic risk – building a product that no longer fits into the overall business strategy for the company
    - Sales risk – building a product that the sales force doesn't understand how to sell
    - Management risk – losing the support of senior management due to a change in focus or a change in people
    - Budget risk – losing budgetary or personnel commitment

- Known risks
    - Those risks that can be uncovered after careful evaluation of the project plan, the business & technical environment in which the project is being developed, & other reliable information sources (e.g. unrealistic delivery date, lack of documented requirements, poor development environment)
- Predictable risks
    - Those risks that are extrapolated from past project experience (e.g. past turnover, poor communication with the customer)
- Unpredictable risks
    - Those risks that can & do occur, but are extremely difficult to identify in advance (e.g. certain changes in government policies may affect the business project)

14) List and explain requirement engineering tasks.

- Requirement
    1. A **function, constraint or other property** that the system must provide to fill the needs of the system's intended user(s).
- Engineering

1. Implies that **systematic and repeatable techniques** should be used.

- Requirement Engineering
  1. It is a systematic approach to **define, manage and test requirements** for a software.
- **Requirements Engineering encompasses seven distinct tasks:**
  1. **Inception (initiation)**
     - **Roughly define scope**
     - Establish a basic understanding of the **problem and the nature of the solution.**
  2. **Elicitation (to gather)**
     - **Define requirements**
     - Draw out the **requirements from stakeholders**.

  3. Elaboration
     - **Further define requirements**, **Expand and refine** requirements obtained from inception & elicitation
     - Create an analysis model that represents **information, functional, and behavioral aspects of the requirements.**

  4. Negotiation
     - Agree on a **deliverable system that is realistic** for developers and customers

  5.Specification (SRS)
     - It may be written document, set of graphical models, formal mathematical model, collection of user scenarios, prototype or collection of these
     - Describe the **requirements formally or informally**.

  6. Validation
     - **Review the requirement specification** for errors, ambiguities, omissions (absence) and conflicts.

  7. Requirements Management
     - It is a set of activities to **identify, control & trace requirements** & changes to requirements (Umbrella Activities) at any time as the project proceeds.

15) Explain Functional Requirement and Non-Functional Requirement with example of Library Management System.

- Functional requirements
  - ✓ Add Book
  - ✓ Update Book
  - ✓ Delete Book
  - ✓ Inquiry Members
  - ✓ Inquiry Issuance
  - ✓ Check out Book
  - ✓ Check In Book
  - ✓ Inquiry waiting for approvals
  - ✓ Reserve Book
- Non-Functional requirements
  - ✓ Safety Requirements
  - ✓ Security Requirements
  - ✓ Software Constraints

- ✓ Response time
- ✓ Availability
- ✓ Reliability
- ✓ Usability
- ✓ Security
- ✓ Capacity
- ✓ Interoperability
- ✓ Utilization

16) Define feasibility study. Enlist and explain the contents to be included in the feasibility study report.

- Establishes the basic business requirements & constraints associated with the application to be built
- Determines whether the solution considered is practical & workable
- Things studied :
    - Resource availability
    - Cost estimation for software development
    - Benefits of the software after development
    - Maintenance cost
- **Types of feasibilities :**
    1. Technical Feasibility
    2. Operational Feasibility
    3. Economic Feasibility

    Technical Feasibility
    - Capability of converting ideas into working systems.
    - Current resources (such as hardware & software) & technology
    - Analyzes the technical skills & capabilities of team
    - Whether the relevant technology is stable
    - Consultation possible if problem arises.

    Operational Feasibility
    - Tests the operational scope of the software to be developed.
    - Assesses extent to which software useful to solve business problems & user requirements
    - Focuses on

    - Whether problems faced are of high priority?
    - Are there acceptable solutions?
    - Will users adapt to new software?
    - Mainly dependent on

    - Development team
    - Visualization of installed operating software

    Economic Feasibility
    - Evaluate the cost of the software development against the ultimate income or benefits
    - Determine whether software is capable of generating financial gains
    - Focuses on

    - Producing long-term gains
    - Cost of requirements elicitation & analysis

- Cost of hardware, software, team & training
    - Involves :
- Cost of development team
- Estimated cost of hardware & software
- Cost of feasibility study

17) List the characteristics of a good quality SRS.

- Software Requirement Specification (SRS) is a document that completely describes what the proposed software should do without describing how software will do it.
- It contains:
    - a complete information description
    - a detailed functional description
    - a representation of system behaviour
    - an indication of performance requirements and design constraints
    - appropriate validation criteria
    - other information suitable to requirements
- SRS is also helping the clients to understand their own needs.

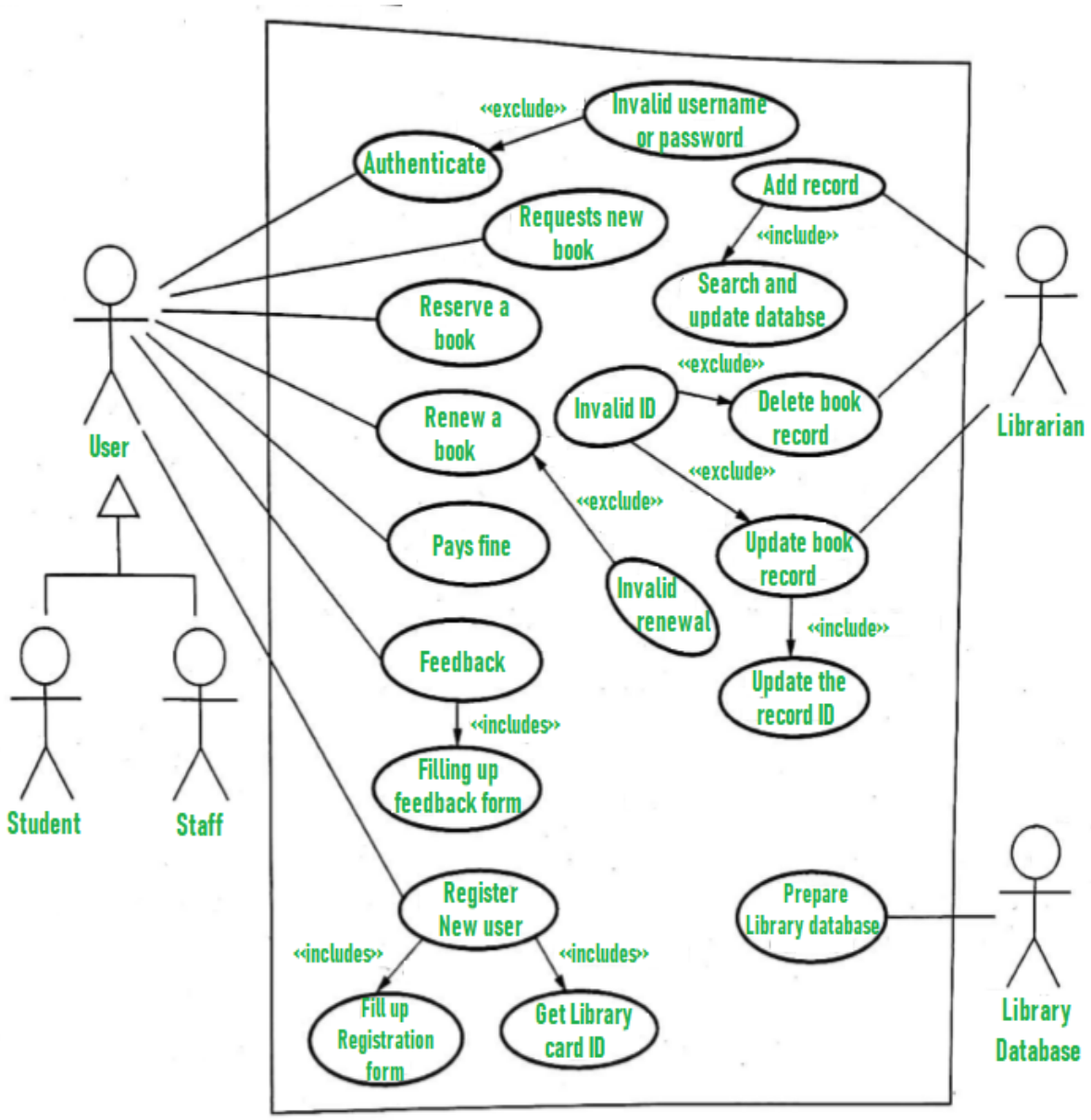
Characteristics of a Good SRS :

- SRS should be accurate, complete, efficient, and of high quality
    - so that it does not affect the entire project plan.
- An SRS is said to be of high quality when the developer and user easily understand the prepared document.
- Characteristics of a Good SRS:
    - **Correct**
        - SRS is correct when all user requirements are stated in the requirements document.
        - Note that there is no specified tool or procedure to assure the correctness of SRS.
    - **Unambiguous**
        - SRS is unambiguous when every stated requirement has only one interpretation.
    - **Complete**
        - SRS is complete when the requirements clearly define what the software is required to do.
    - **Ranked for Importance/Stability**
        - All requirements are not equally important, hence each requirement is identified to make differences among other requirements.
        - Stability implies the probability of changes in the requirement in future.
    - **Modifiable**
        - The requirements of the user can change, hence requirements document should be created in such a manner that those changes can be modified easily.
    - **Traceable**
        - SRS is traceable when the source of each requirement is clear and facilitates the reference of each requirement in future.
    - **Verifiable**
        - SRS is verifiable when the specified requirements can be verified with

a cost-effective process to check whether the final software meets those requirements.

- **Consistent**
  - SRS is consistent when the subsets of individual requirements defined do not conflict with each other.

18) Draw Use case for Library Management System.

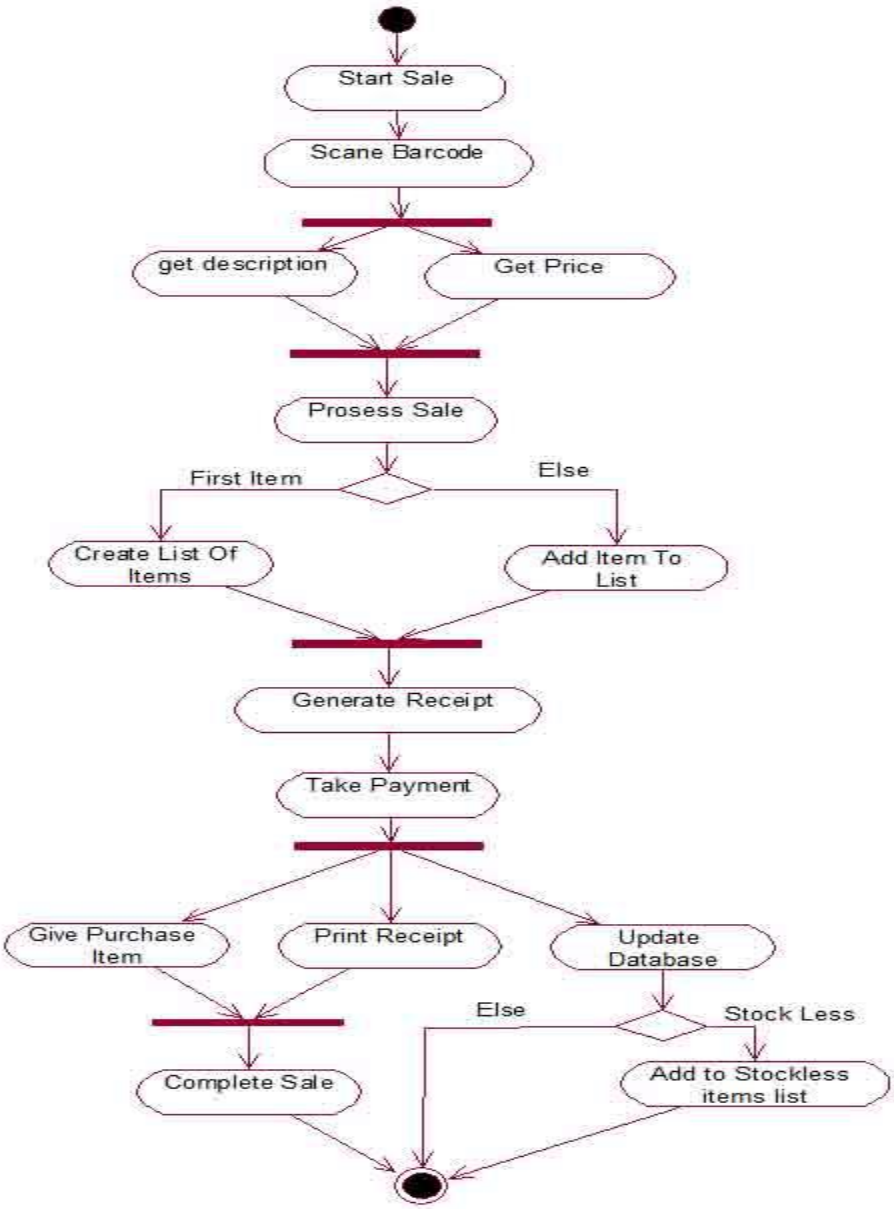


19) What is activity diagram and swim-lane diagram? Draw activity diagram for Billing Counter of a shopping mall.

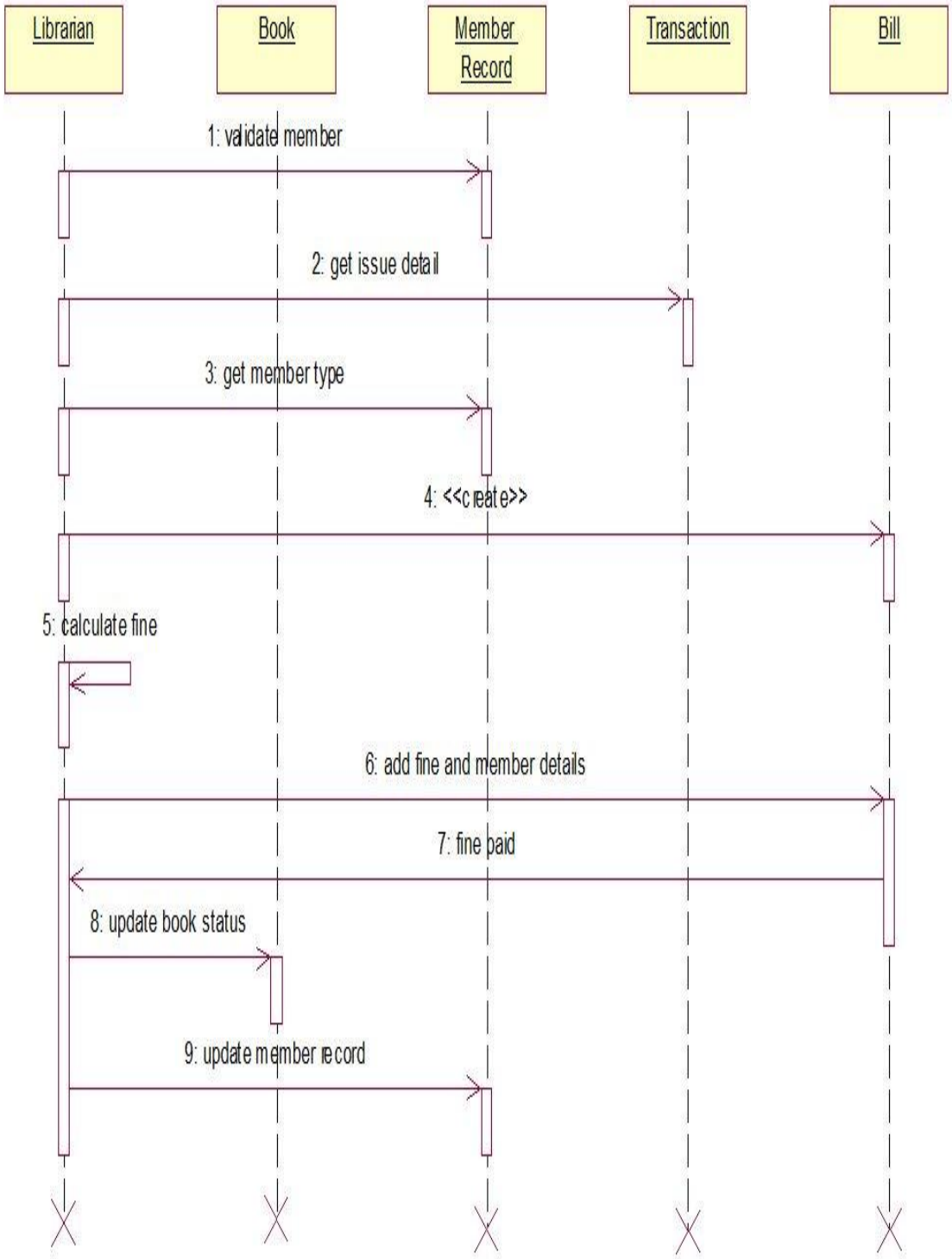
The **Activity diagrams** in Object Oriented Design are just like the flow charts that show the sequence of steps that make up a complex process, such as an algorithm or workflow. Activity diagrams are most useful during the initial stages of the design phase.

Swim Lanes:

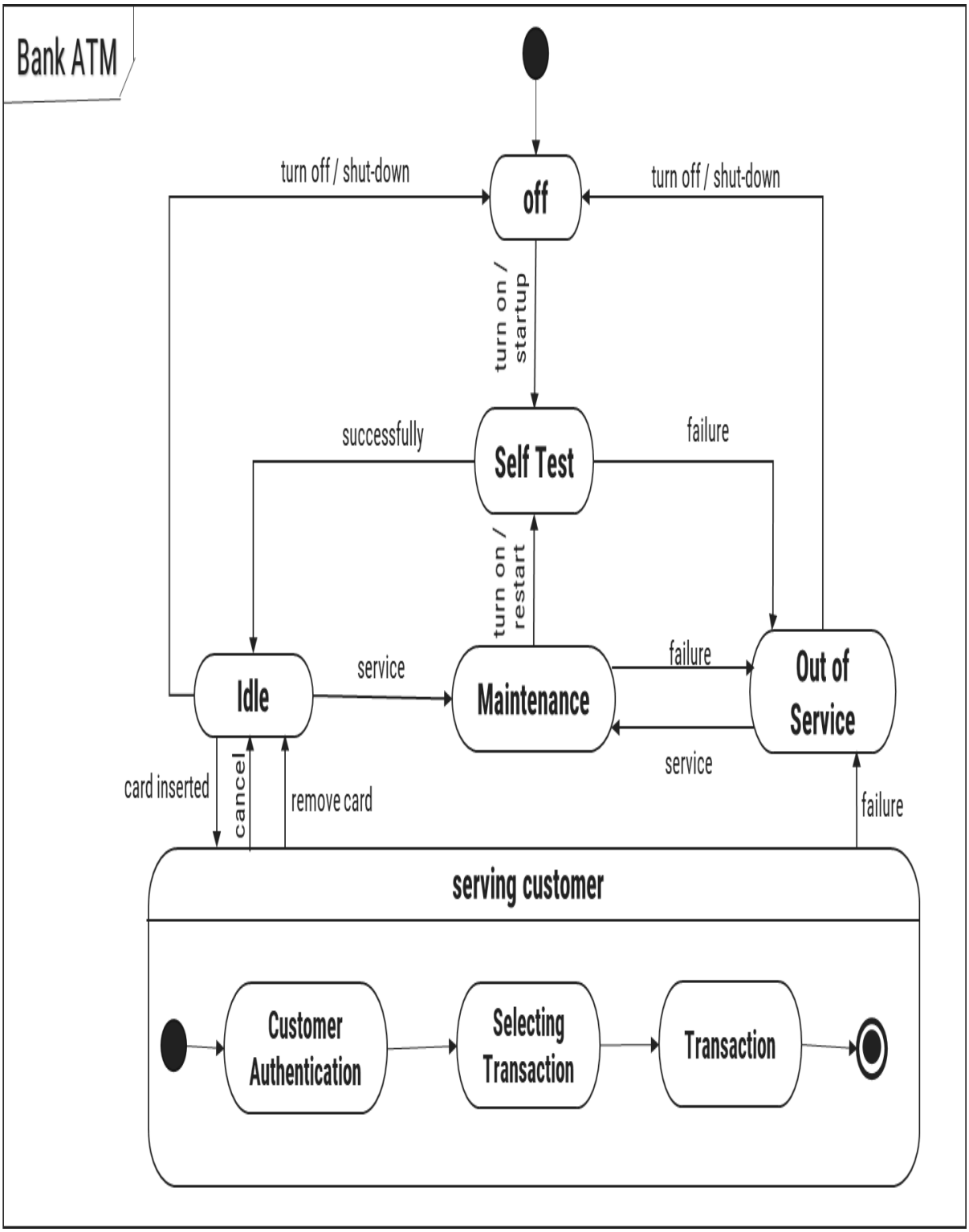
Swimlanes are used to show which activities are performed by which organisation in the activity diagram. The lanes are boundaries are drawn and the activities of a particular organisation are drawn in the same lane as that of the organisation. Swimlanes have to be ordered in a Logical Manner. It is suggested to have less than five swimlanes in an activity diagram. Swimlanes are good in that they combine the activity diagram's depiction of logic with the interaction diagram's depiction of responsibility.



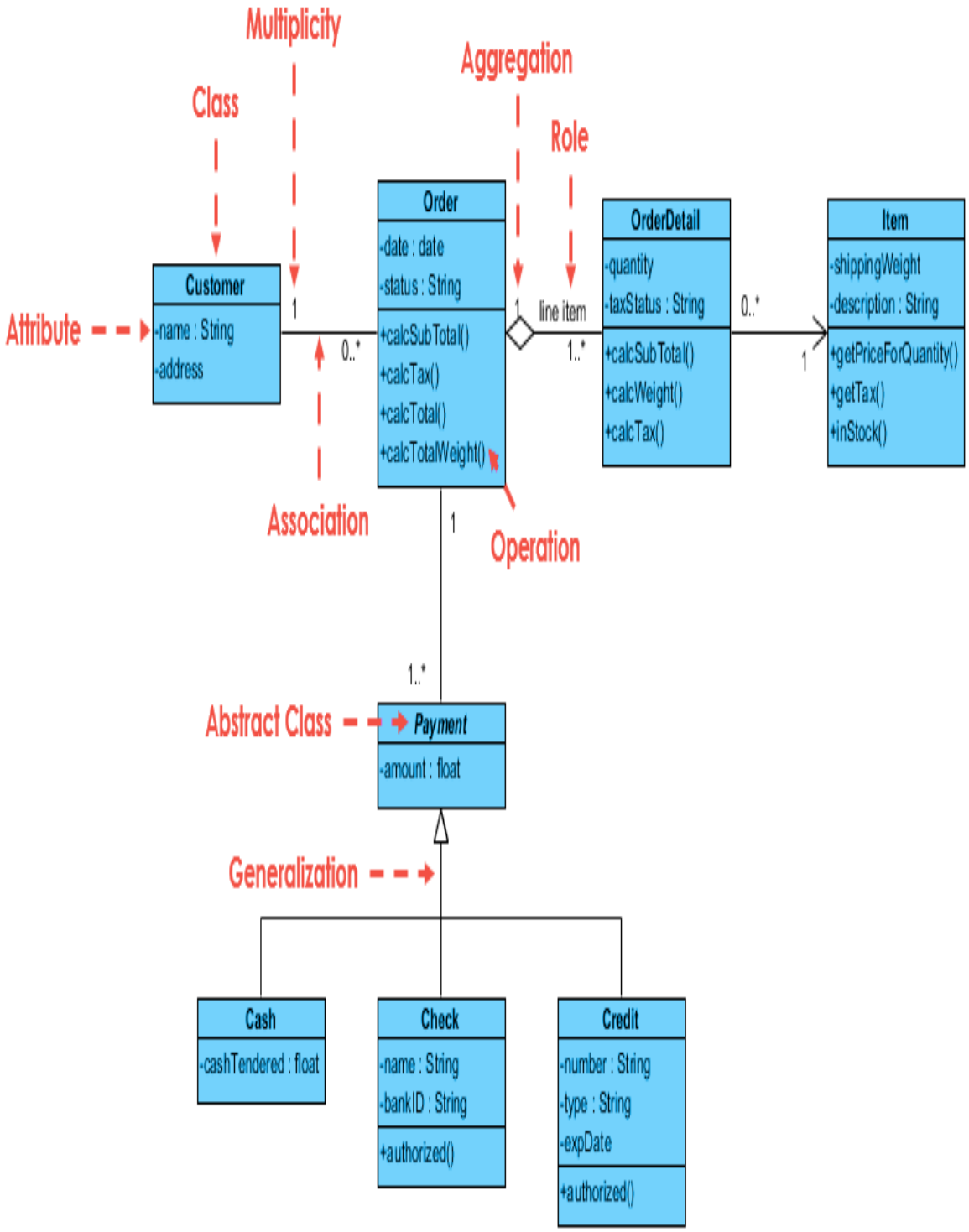
20) Draw Sequence Diagram for Library Management System.



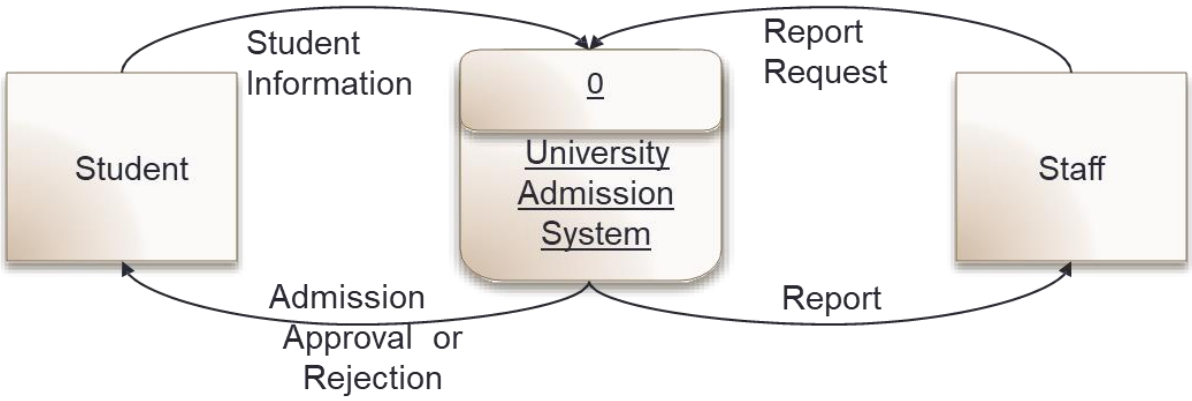
21) Draw State diagram of Bank ATM



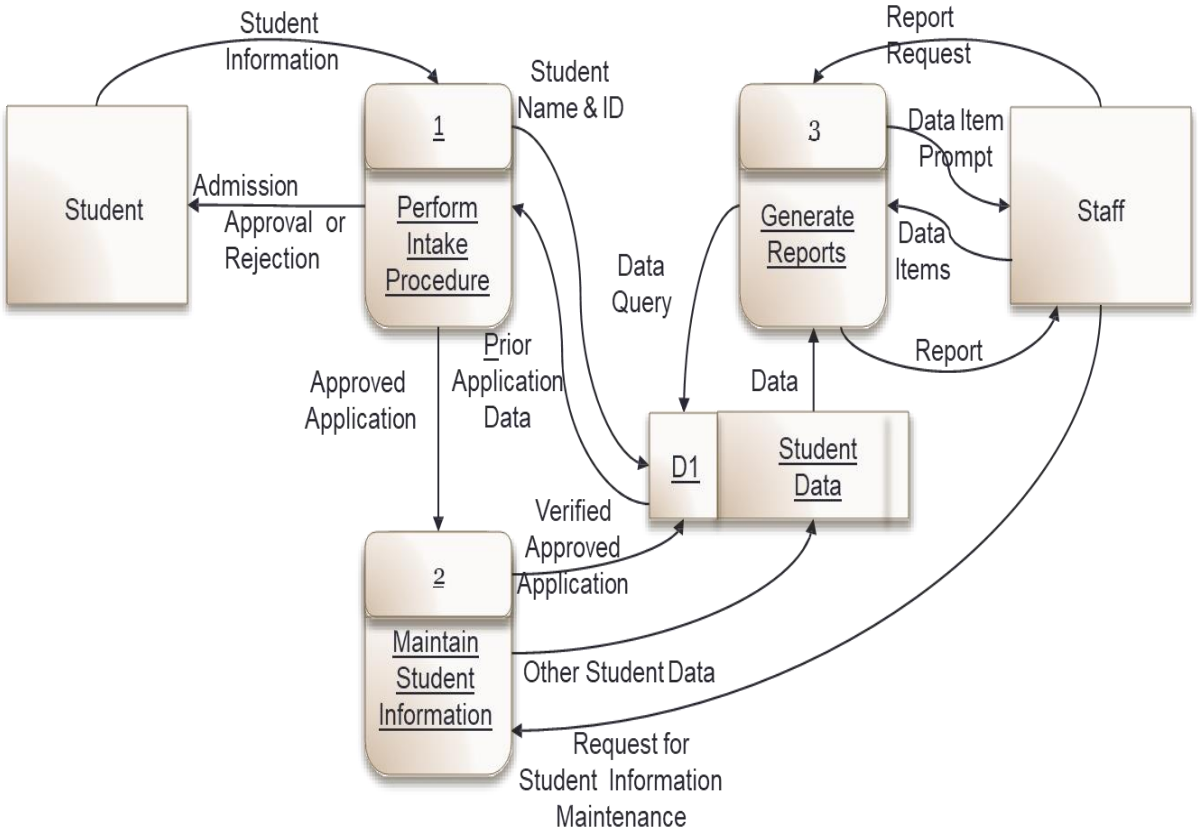
22) Draw Class diagram of Order System


Multiplicity
Class
Aggregation
Role
Attribute
Association
Operation
Abstract Class
Generalization
Customer
-name : String
-address
Order
-date : date
-status : String
+calcSubTotal()
+calcTax()
+calcTotal()
+calcTotalWeight()
OrderDetail
-quantity
-taxStatus : String
+calcSubTotal()
+calcWeight()
+calcTax()
Item
-shippingWeight
-description : String
+getPriceForQuantity()
+getTax()
+inStock()
line item
1
0..*
1..*
Payment
-amount : float
Cash
-cashTendered : float
Check
-name : String
-bankID : String
+authorized()
Credit
-number : String
-type : String
-expDate
+authorized()

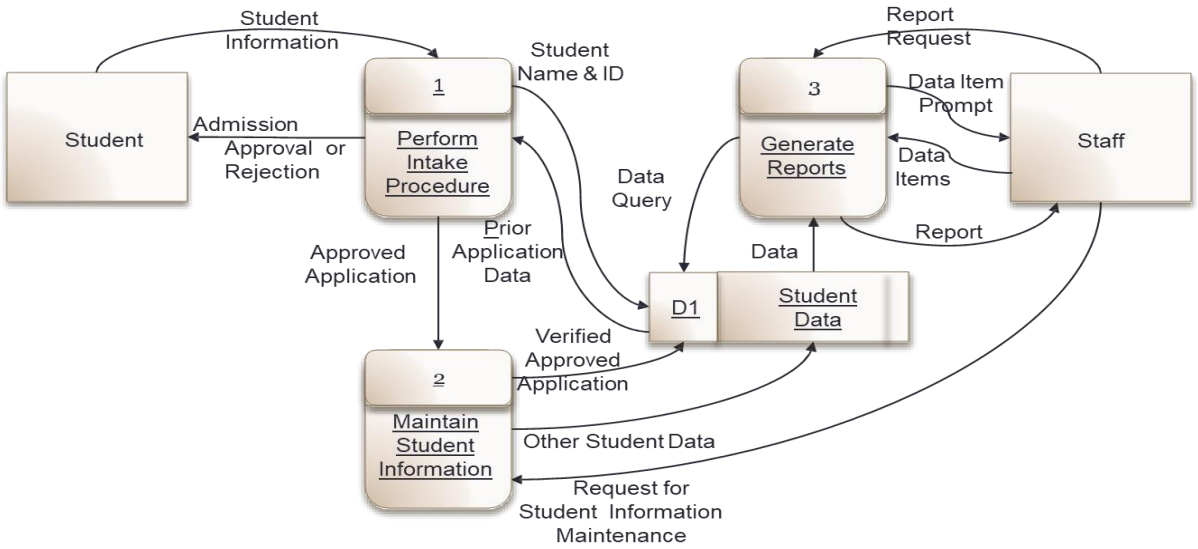
23) Draw DFD for University Admission System



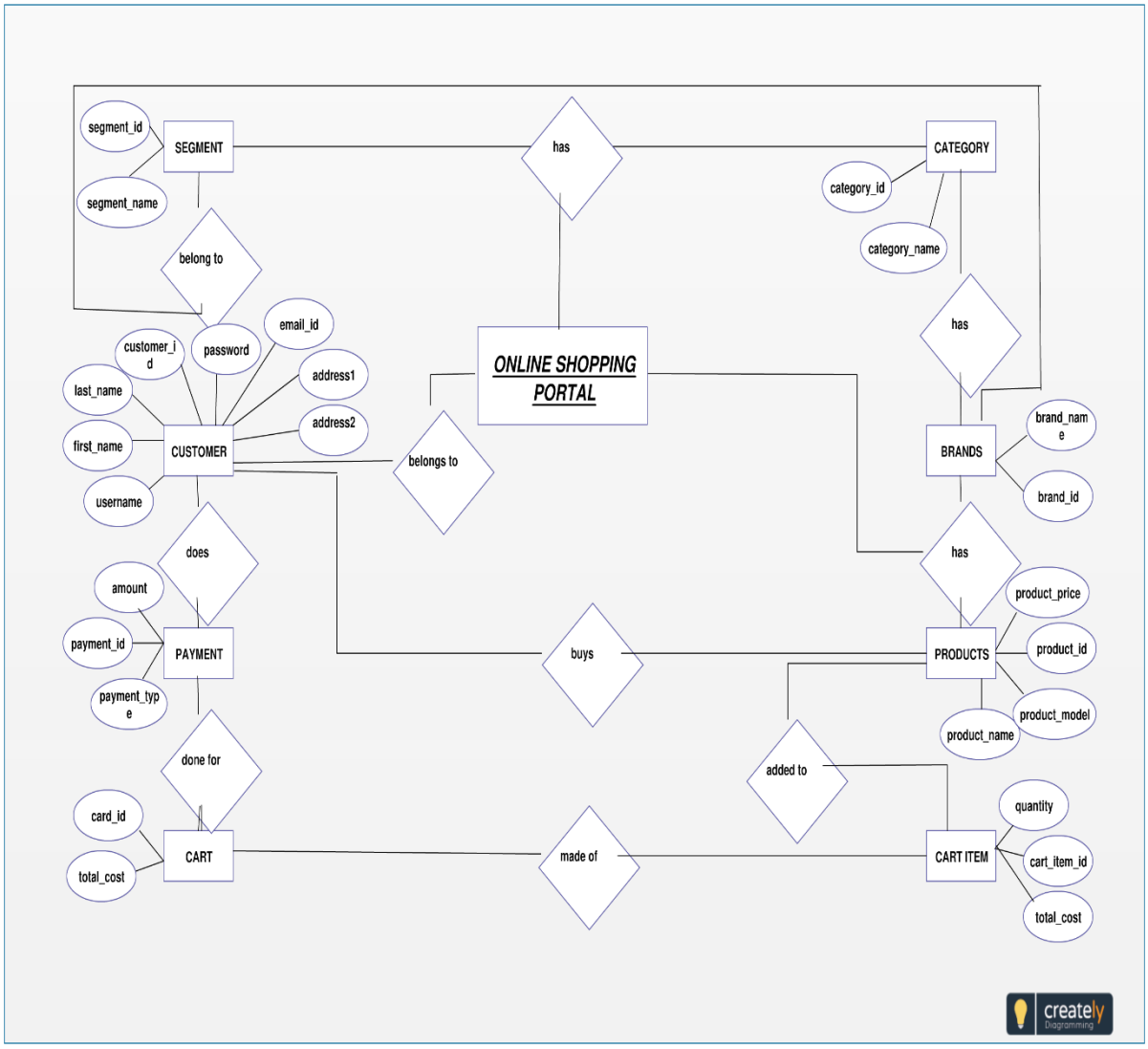
Level 1



Level 2

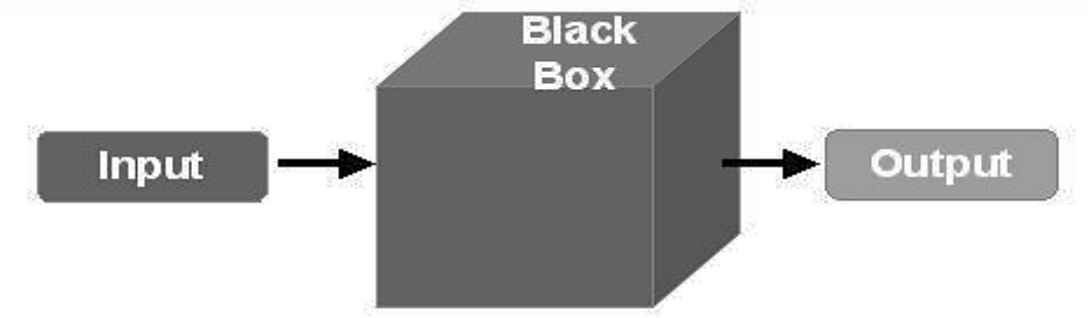


24) Draw E-R Diagram for Online shopping System



25) Explain Black box testing. Explain any one technique.

Black Box Testing :



Black Box testing :

- Black box testing is the software testing method which is used to test the software without knowing the internal structure of code or program.
- It is also known as functional testing & specification-based testing.
- Basically software under test is called as "Black-Box", we are treating this as black box & without checking internal structure of software we test the software.
- In Black Box Testing we just focus on inputs and output of the software system without bothering about internal knowledge of the software program.
- Tester has access only to running code and the specification it is supposed to satisfy.
- Test cases are written with no knowledge of internal workings of the code.
- No access to source code.
- The above Black Box executable program can be any software system you want to test.
- For example : an operating system like Windows, a website like Google ,a database like Oracle or even your own custom application. Under Black Box Testing , you can test these applications by just focusing on the inputs & outputs without knowing their internal code implementation.

Advantages :

- More effective on larger units of code than glass box testing.
- Tester needs no knowledge of implementation, including specific programming languages.
- Tester and programmer are independent of each other.
- Tests are done from a user's point of view.
- Will help to expose any ambiguities or inconsistencies in the specifications.
- Test cases can be designed as soon as the specifications are complete.

Disadvantages :

- Only a small number of possible inputs can actually be tested, to test every possible input stream would take nearly forever.
- Without clear and concise specifications, test cases are hard to design.
- There may be unnecessary repetition of test inputs if the tester is not informed of test cases the programmer has already tried.
- May leave many program paths untested.

Equivalence partitioning:

- Equivalence Partitioning also called as equivalence class partitioning.

- It can be applied at any level of testing and is often a good technique to use first.
- The idea behind this technique is to divide (i.e. to partition) a set of test conditions into groups or sets that can be considered the same (i.e. the system should handle them equivalently), hence 'equivalence partitioning'.
- In equivalence partitioning technique we need to test only one condition from each partition. This is because we are assuming that all the conditions in one partition will be treated in the same way by the software.
- Equivalence partitioning is a testing technique where input values set into classes for testing.

Valid Input Class = Keeps all valid inputs.
Invalid Input Class = Keeps all Invalid inputs.

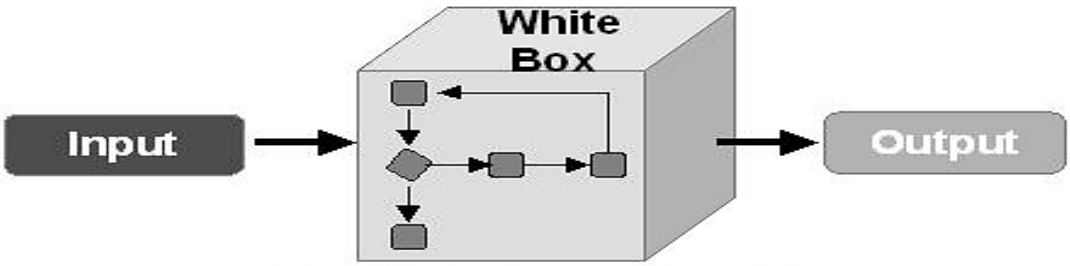
Equivalence classes may be defined according to the following guidelines:
1. If an input condition specifies a range, one valid and two invalid equivalence classes are defined.
2. If an input condition requires a specific value, one valid and two invalid equivalence classes are defined.
3. If an input condition specifies a member of a set, one valid and one invalid equivalence class are defined.
4. If an input condition is Boolean, one valid and one invalid class are defined.

- By applying the guidelines for the derivation of equivalence classes, test cases for each input domain data item can be developed and executed.

- A text field permits only numeric characters
- Length must be 6-10 characters long
- Partition according to the requirement should be like this:
- While evaluating Equivalence partitioning, values in all partitions are equivalent that's why 0-5 are equivalent, 6 – 10 are equivalent and 11- 14 are equivalent.
- At the time of testing, test 4 and 12 as invalid values and 7 as valid one.
- It is easy to test input ranges 6–10 but harder to test input ranges 2-600. Testing will be easy in the case of lesser test cases but you should be very careful.
- Assuming, valid input is 7. That means, you belief that the developer coded the correct valid range (6-10).

26) Explain White box testing. Explain any one technique.

White box testing



- White box testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is known to the tester.
- It is like the work of a mechanic who examines the engine to see why the car is not moving.
- Focus is on ensuring that even abnormal invocations are handled gracefully.

- Using white-box testing methods, you can derive test cases that
    - Guarantee that all independent paths within a module have been exercised at least once
    - Exercise all logical decisions on their true and false sides
    - Execute all loops at their boundaries
    - Exercise internal data structures to ensure their validity
- Why is it called White box testing ? - Because here the testers require knowledge of how the software is implemented.
- It is applicable to the following levels of software testing
    - Unit Testing: For testing paths within a unit
    - Integration Testing: For testing paths between units
    - System Testing: For testing paths between subsystems
- White box testing is mainly used for detecting logical errors in the program code.
- It is used for debugging a code, finding random typographical errors, and uncovering incorrect programming assumptions.
- White box testing is done at low level design and implementable code.
- It can be applied at all levels of system development especially Unit, system and integration testing.
- White box testing can be used for other development artifacts like requirements analysis, designing and test cases.

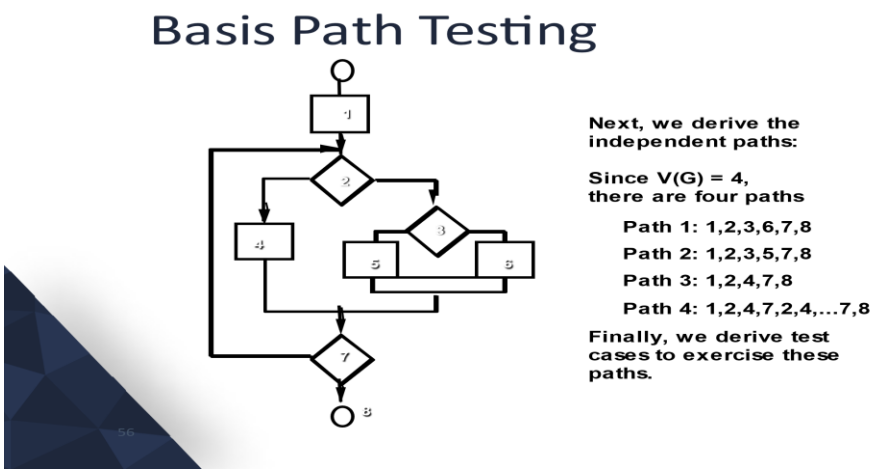
Advantages :

- Reveals errors in "hidden" code.
- Spots the Dead Code or other issues with respect to best programming practices.
- Testing is more thorough, with the possibility of covering most paths.

Disadvantages :

- Expensive as one has to spend both time and money to perform white box testing.
- Every possibility that few lines of code are missed accidentally.
- In-depth knowledge about the programming language is necessary to perform white box testing.
- Since tests can be very complex, highly skilled resources are required, with thorough knowledge of programming and implementation.

Whitebox Testing Techniques:



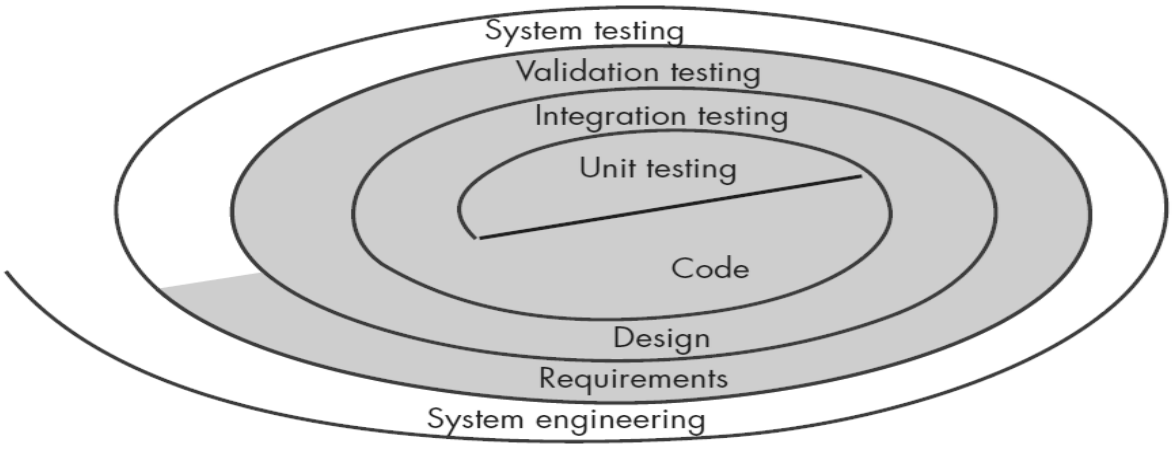
27) Discuss the differences between black box and white box testing.

Black Box Testing	White Box Testing
It is a way of software testing in which the internal structure or the program or the code is hidden and nothing is known about it.	It is a way of testing the software in which the tester has knowledge about the internal structure or the code or the program of the software.
Implementation of code is not needed for black box testing.	Code implementation is necessary for white box testing.
It is mostly done by software testers.	It is mostly done by software developers.
No knowledge of implementation is needed.	Knowledge of implementation is required.
It can be referred to as outer or external software testing.	It is the inner or the internal software testing.
It is a functional test of the software.	It is a structural test of the software.
This testing can be initiated based on the requirement specifications document.	This type of testing of software is started after a detail design document.
No knowledge of programming is required.	It is mandatory to have knowledge of programming.
It is the behavior testing of the software.	It is the logic testing of the software.
It is applicable to the higher levels of testing of software.	It is generally applicable to the lower levels of software testing.
It is also called closed testing.	It is also called as clear box testing.
It is least time consuming.	It is most time consuming.
It is not suitable or preferred for algorithm testing.	It is suitable for algorithm testing.
Can be done by trial and error ways and methods.	Data domains along with inner or internal boundaries can be better tested.

Black Box Testing	White Box Testing
Example: Search something on google by using keywords **Black-box test design techniques-** • Decision table testing • All-pairs testing • Equivalence partitioning • Error guessing	**Example:** By input to check and verify loops **White-box test design techniques-** • Control flow testing • Data flow testing • Branch testing
Types of Black Box Testing: • Functional Testing • Non-functional testing • Regression Testing	**Types of White Box Testing:** • Path Testing • Loop Testing • Condition testing
It is less exhaustive as compared to white box testing.	It is comparatively more exhaustive than black box testing.

28) Explain different levels of testing and the goals of the different levels.

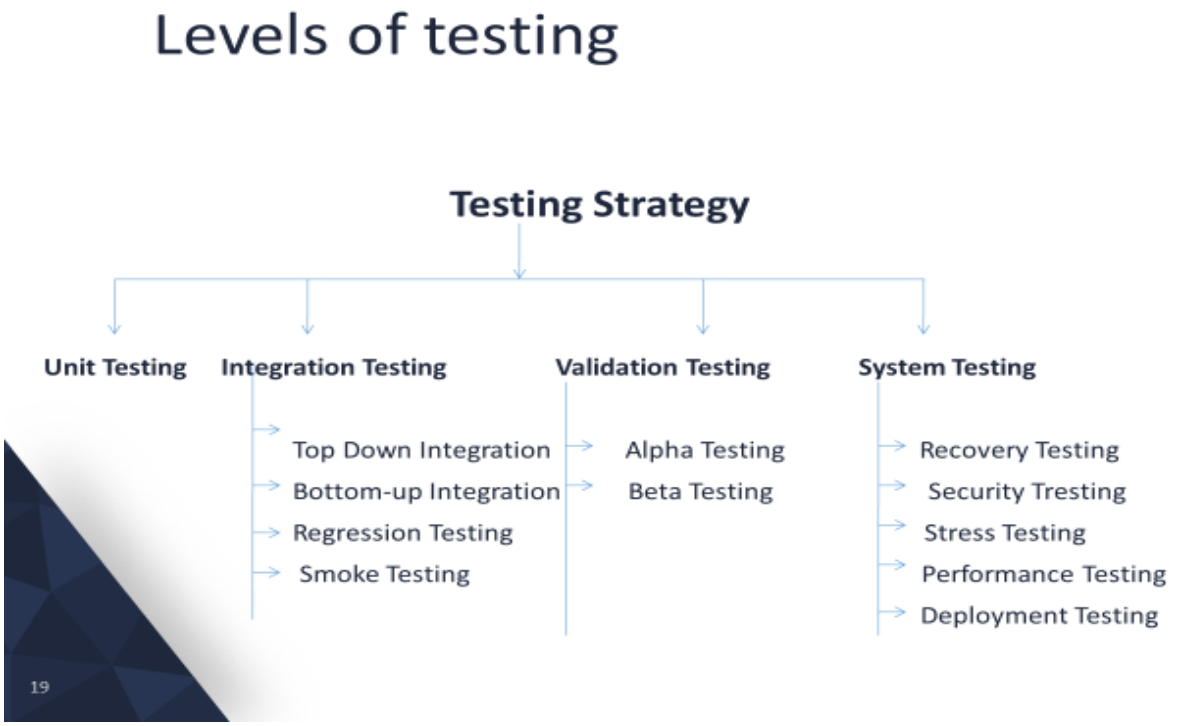
- We begins by 'testing-in-the-small' & move toward 'testing-in-the-large'.
- Various testing strategies for conventional software are
  1. Unit testing
  2. Integration testing
  3. Validation testing
  4. System testing



Testing Strategy :

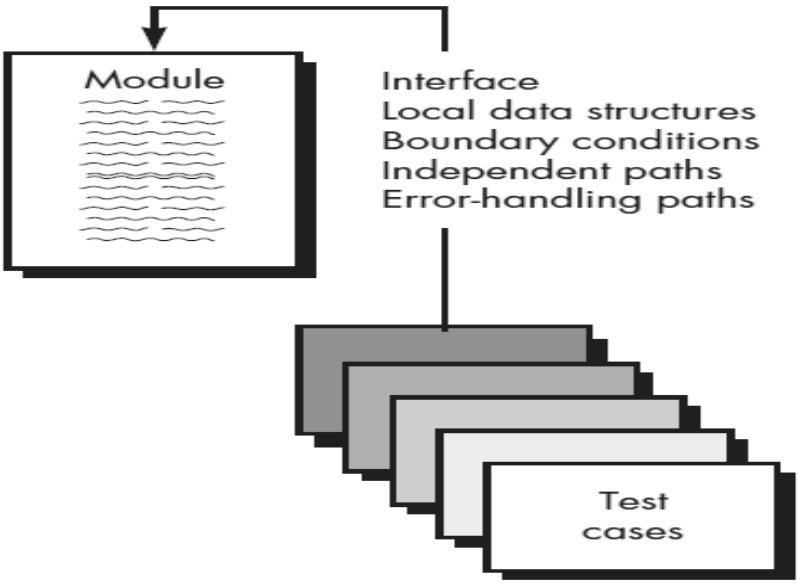
- **Unit testing –** In this type of testing techniques are applied to detect the errors from each software component individually.

- **Integration testing –** It focuses on issues associated with verification & program construction as components begin interacting with one another.
- **Validation testing –** It provides assurance that the software validation criteria meets all functional, behavioral & performance requirements.
- **System testing –** In system testing all system elements forming the system is tested as a whole.



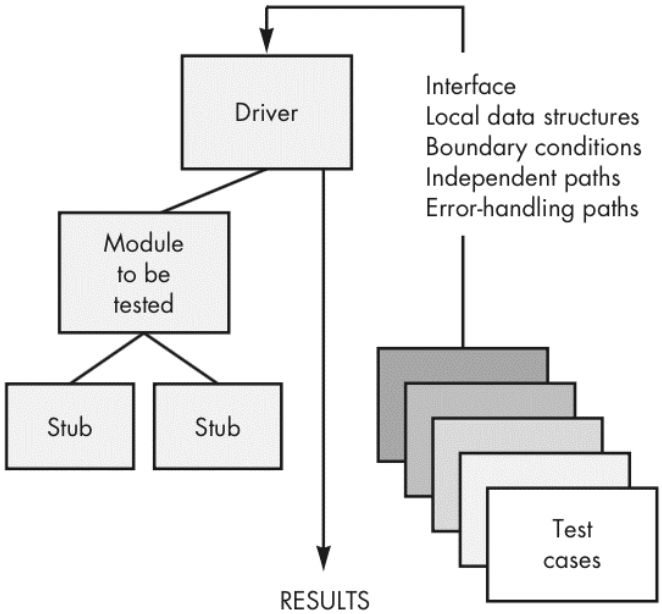
Unit testing :

- Unit is the smallest part of a software system which is testable.
- It may include code files, classes and methods which can be tested individually for correctness.
- It concentrate on each unit of the software as implemented in source code.
- It focuses on each component individual, ensuring that it functions properly as a unit.
- The unit test focuses on the internal processing logic and data structures within the boundaries of a component.



The various tests that are conducted during the unit test are described as :

- Module interfaces are tested for proper information flow in and out of the program unit.
- Local data are examined to ensure that integrity is maintained.
- Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restricted processing.
- All the independent paths are tested for ensuring that all statements in the module have been executed only once.
- All error handling paths should be tested.
- Drivers & stub software need to be developed to test incomplete software.
- The unit testing is simplified when a component with high cohesion is designed.



29) Distinguish between verification and validation.

Verification	Validation
It includes checking documents, design, codes and programs.	It includes testing and validating the actual product.
Verification is the static testing.	Validation is the dynamic testing.
It does *not* include the execution of the code.	It includes the execution of the code.
Methods used in verification are reviews, walkthroughs, inspections and desk-checking.	Methods used in validation are Black Box Testing, White Box Testing and non-functional testing.
It checks whether the software conforms to specifications or not.	It checks whether the software meets the requirements and expectations of a customer or not.
It can find the bugs in the early stage of the development.	It can only find the bugs that could not be found by the verification process.
The goal of verification is application and software architecture and specification.	The goal of validation is an actual product.
Quality assurance team does verification.	Validation is executed on software code with the help of testing team.
It comes before validation.	It comes after verification.
It consists of checking of documents/files and is performed by human.	It consists of execution of program and is performed by computer.
Verification refers to the set of activities that ensure software correctly implements the specific function.	Validation refers to the set of activities that ensure that the software that has been built is traceable to customer requirements.
After a valid and complete specification the verification starts.	Validation begins as soon as project starts.
Verification is for prevention of errors.	Validation is for detection of errors.

Verification	Validation
Verification is also termed as white box testing or static testing as work product goes through reviews.	Validation can be termed as black box testing or dynamic testing as work product is executed.
Verification finds about 50 to 60% of the defects.	Validation finds about 20 to 30% of the defects.
Verification is based on the opinion of reviewer and may change from person to person.	Validation is based on the fact and is often stable.
Verification is about process, standard and guideline.	Validation is about the product.

30) Define Quality for software. List and explain SQA activities.

Software quality:

- ***software quality*** measures how well software is designed (quality of design), and how well the software conforms to that design (quality of conformance)

Software Quality Assurance (SQA) :

- Software quality assurance (also called quality management) is an umbrella activity that is applied throughout the software process.
- It is planned and systematic pattern of activities necessary to provide high degree of confidence in the quality.
- Software quality assurance (SQA) have:
    - An SQA process.
    - Specific quality assurance and quality control tasks.
    - Control of all software work products.
    - A procedure to ensure compliance with software development standards.

    Software measurement mechanisms.

Software Quality Assurance (SQA) Activities :

- SQA tasks are associated with –
    - Software engineers (Responsible for developing the product)
    - SQA group (Responsible for performing QA planning, record keeping, analysis & reporting)
- SQA activities conducted by SQA group are :
    - Create a SQA plan.
    - Participates in description of software process.
    - Reviews software engineering activities.
    - Authenticate designated software work products.
    - Ensure that deviations in software work and work products are documented and

handled according to a documented procedure.
    - Identify any noncompliance & reports to senior management.

Importance of SQA :

- Quality control and assurance are essential activities for any business that produces products to be used by others.
- Software quality assurance starts from the beginning of a project, right from the analysis phase.
- The major reason of involving quality assurance in software product development is
    - to make sure that the final product built is as per the requirement specification and comply with the standards.

31) Write a short note on formal technical review.

- A formal technical review (FTR) **is a software quality control activity** performed by software engineers (and others).
- **The objectives of an FTR are:**
- To **uncover errors** in function, logic, or implementation; for any representation of the software.
- To verify that the software under review meets its requirements.
- To ensure that the software has been represented according to predefined standards.
- To achieve software that is developed in a uniform manner.
- To make projects more manageable.
- A formal technical review (FTR) is a software quality control activity performed by software engineers (and others).
- The objectives of an FTR are :
    - To uncover errors in function, logic, or implementation; for any representation of the software
    - To verify that the software under review meets its requirements
    - To ensure that the software has been represented according to predefined standards
    - To achieve software that is developed in a uniform manner
    - To make projects more manageable
- During the FTR,
    - a reviewer (the recorder) actively records all issues that have been raised.
- These are summarized at the end of the review meeting, and a reviewed issues list is produced.
- In addition, a formal technical review summary report is completed.

Review Guidelines :

- Guidelines for the conducting of formal technical reviews must be established in advance.
- It must be distributed to all reviewers, agreed upon & then followed.
- Guideline for review may include following things –
    - Concentrate on work product only.
    - Set an agenda of review & maintain it.
    - When certain issues are raised then debate or arguments should be limited.

- Find out problem areas.
- Take written notes.
- Limit the numbers of participants & insist upon advance preparation.
- Develop a checklist for each product that is likely to be reviewed.
- Allocate resources & time schedule for FTRs.
- Conduct meaningful training for all reviewers.
- Review earlier reviews which serves as the base for the current review being conducted.

32) List quality standards. Explain anyone.

List quality standards:

- ISO 9000 and 9001
- ISO 9001:2000
- Six Sigma
- CMM

SEI -CMM (Software Engineering Institute-Capability Maturity Model)

- To determine an organization's current state of process maturity, the SEI (Software Engineering Institute) uses an assessment that results in a five point grading scheme.
- The grading scheme determines compliance with a capability maturity model (CMM) that defines key activities required at different levels of process maturity.
- The SEI approach establishes five process maturity levels that are defined in the following manner.

Level 1: Initial

- The software process is characterized as ad hoc and occasionally.
- Few processes are defined and success depends on individual effort.

Level 2: Repeatable

- Basic project management processes are established to track cost, schedule, and functionality.
- The necessary process discipline is in place to repeat earlier successes on project.

Level 3: Defined

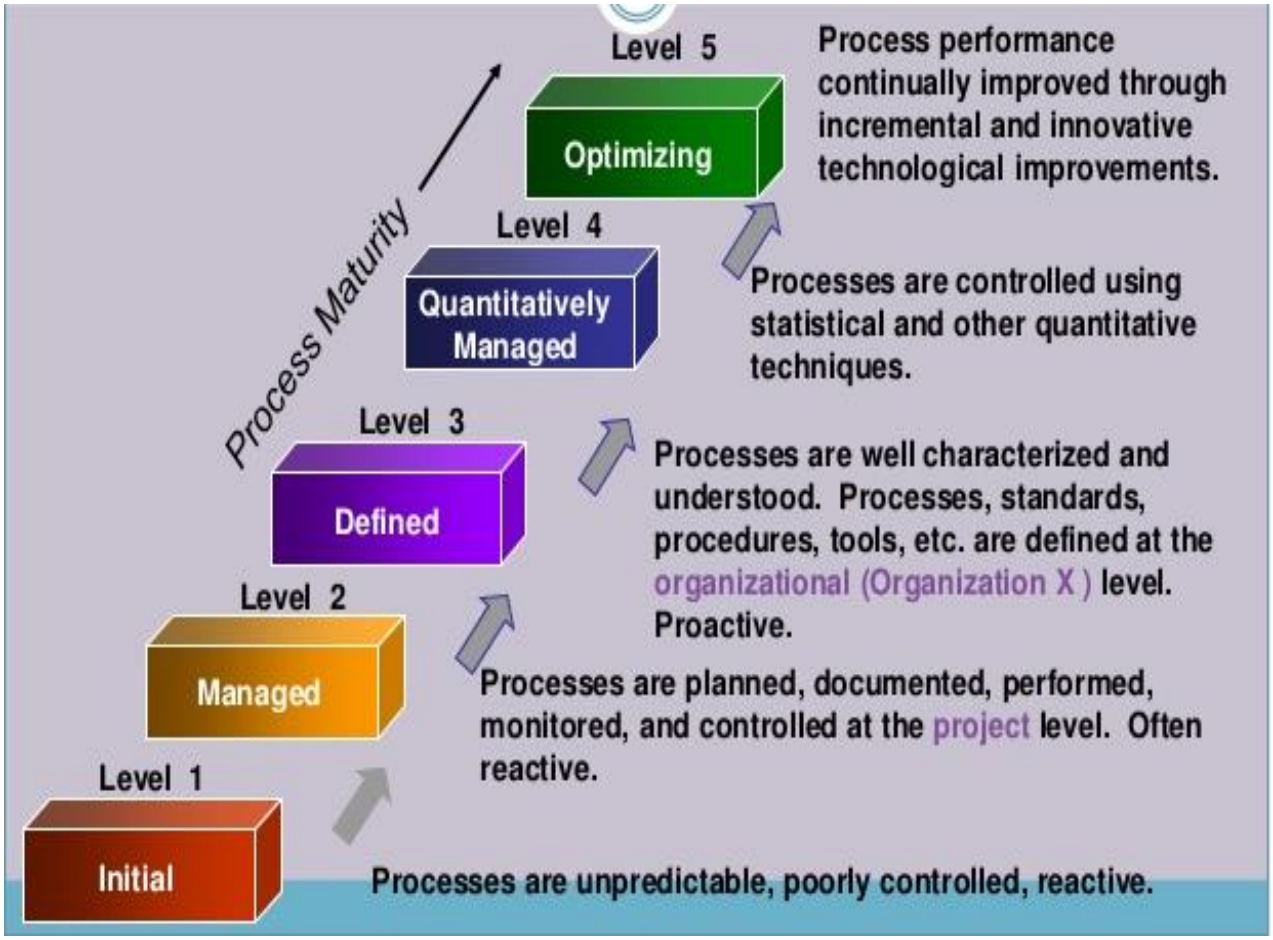
- The software process for both management and engineering activities is documented, standardized and integrated.
- This level includes all characteristics defined for level 2.

Level 4: Managed

- Detailed measures of the software process and product quality are collected.
- This level includes all characteristics defined for level 3.

Level 5: Optimizing

- Continuous process improvement is enabled by quantitative feedback from the process and from testing innovative ideas and technologies.
- This level includes all characteristics defined for level 4.



33) Explain Software Configuration Management (SCM) process in detail.

SCM (Software Configuration Management) :

- The SCM is a set of activities that have been **developed to manage change throughout the software life cycle.**
- **Purpose**
  - Systematically control changes to the configuration and maintain the integrity and traceability of the configuration throughout the system's life cycle.
- The software configuration management process **defines a series of tasks that have four primary objectives:**
  - **To identify all items** that collectively define the software configuration.
  - **To manage changes** to one or more of these items.
  - To facilitate the **construction of different versions** of an application.
  - To ensure that software quality is maintained as the configuration evolves over time.

- **SCIs (Software Configuration Item)** flow outward through these layers throughout their useful life.
- As an SCI moves through a layer, the actions implied by each SCM task may or may not be applicable.
    - For example, when a new SCI is created, it must be identified.
    - However, if no changes are requested for the SCI, the change control layer does not apply.
- The SCI is assigned to a specific version of the software (version control mechanisms come into play).
- A record of the SCI (its name, creation date, version, etc.) is maintained for configuration auditing purposes.

Identification task :

- **Identification** separately names each SCI and then organizes it in the SCM repository using an object-oriented approach.
- Objects start out as basic objects and are then grouped into aggregate objects.
- Each object has a set of distinct features that identify it.
    - **A name** that is unambiguous to all other objects
    - **A description** that contains the SCI type, a project identifier, and change and/or version information
    - **List of resources** needed by the object

Change Control (CCB) :

- Change control is a procedural activity that ensures quality and consistency as changes are made to a configuration object.
- A change request is submitted to a configuration control authority, which is usually a change control board (CCB).
    - The request is evaluated for technical merit, potential side effects, overall impact on other configuration objects and system functions, and projected cost in terms of money, time and resources.

- An engineering change order (ECO) is issued for each approved change request.
    - Describes the change to be made, the constraints to follow and the criteria for review and audit.
- The Baselined SCI is obtained from the SCM repository.
    - Access control governs which software engineers have the authority to access and modify a particular configuration object.
    - Synchronization control helps to ensure that parallel changes performed by two different people don't overwrite one another.

Version Control :

- Version control is a set of procedures and tools for managing the creation and use of multiple occurrences of objects in the SCM repository.
- Version Control Capabilities
    - An SCM repository that stores all relevant configuration objects.
    - A version management capability that stores all versions of a configuration object.
    - A make facility that enables the software engineer to collect all relevant configuration objects and construct a specific version of the software.
    - Issues tracking (bug tracking) capability that enables the team to record and track the status of all outstanding issues associated with each configuration object.
- The SCM repository maintains a change set.
    - Serves as a collection of all changes made to a baseline configuration.
    - Used to create a specific version of the software.
    - Captures all changes to all files in the configuration along with the reason for changes and details of who made the changes and when.

Configuration Audit :

- Configuration auditing is an SQA activity that helps to ensure that quality is maintained as changes are made.
- It complements the formal technical review and is conducted by the SQA group.
- **It addresses the following questions**
    - Has a formal technical review been conducted to assess technical correctness?
    - Has the software process been followed and have software engineering standards been properly applied?
    - Has the change been "highlighted" and "documented" in the SCI? Have the change data and change author been specified? Do the attributes of the configuration object reflect the change?
    - Have SCM procedures for noting the change, recording it and reporting it been followed?
    - Have all related SCIs been properly updated?
- A configuration audit ensures that
    - The correct SCIs (by version) have been incorporated into a specific build.
    - That all documentation is up-to-date and consistent with the version that has been built.

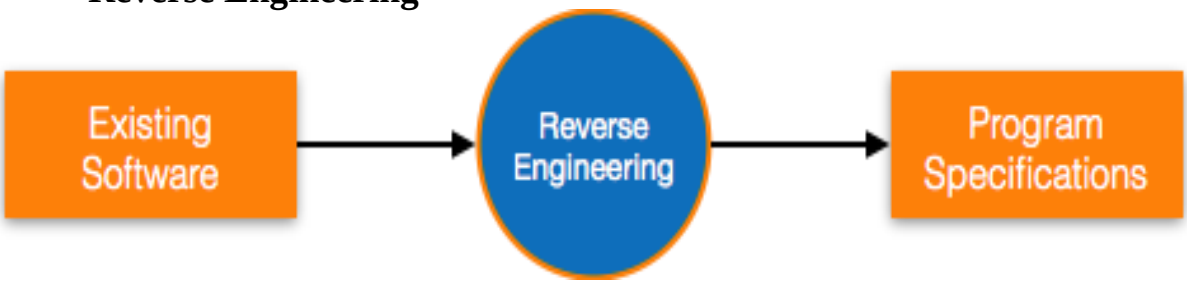
Status Reporting :

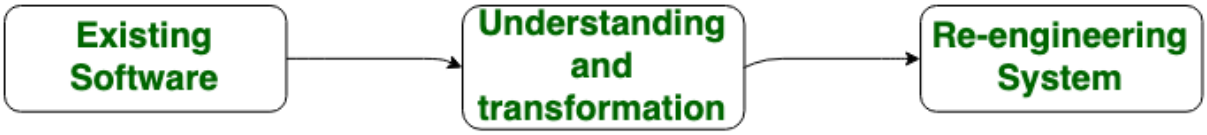
- Configuration status reporting (CSR) is also called status accounting.

- Provides information about each change to those personnel in an organization with a need to know.
- Answers what happened, who did it, when did it happen and what else will be affected?
- Sources of entries for configuration status reporting
    - Each time a SCI is assigned new or updated information
    - Each time a configuration audit is conducted

34) Write short note on: Reverse Engineering.

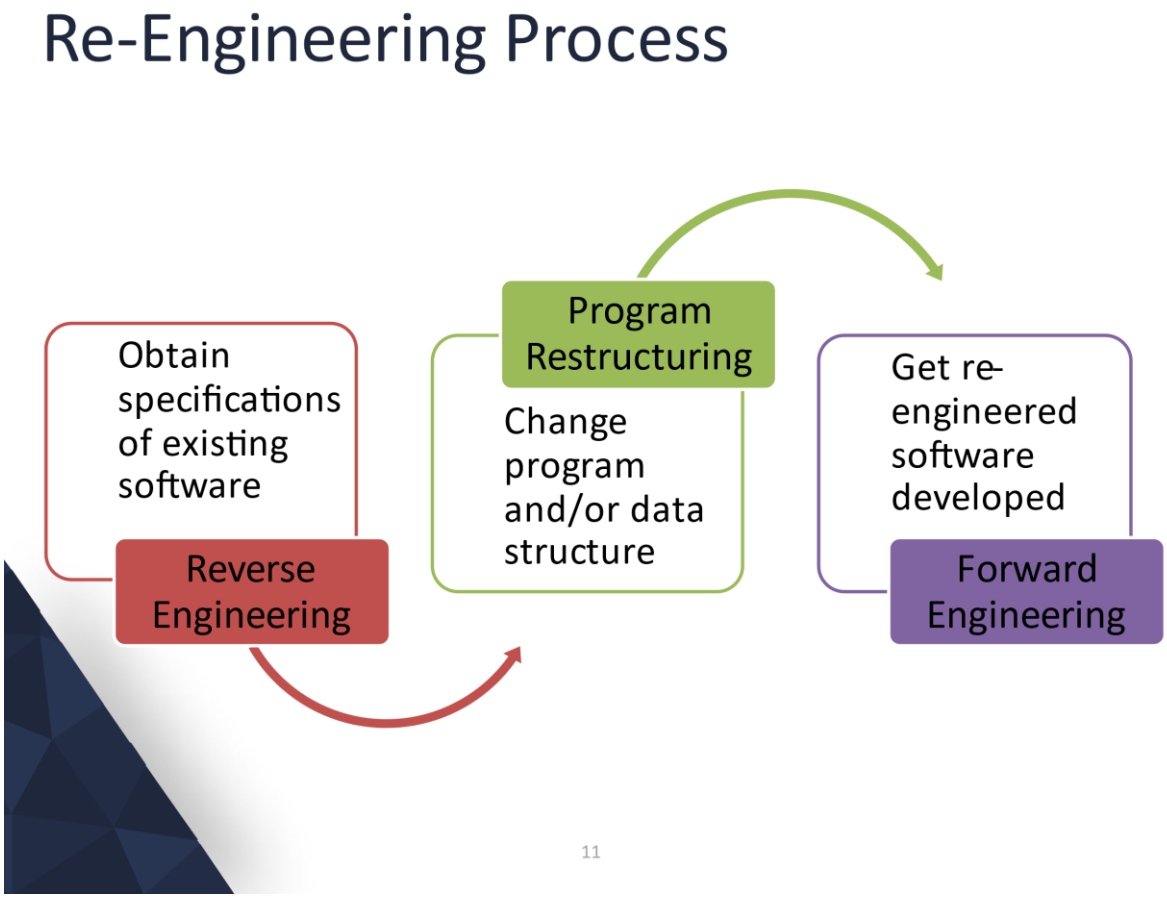
- **Reverse Engineering**





Reverse Engineering

- It is a process to achieve system specification by thoroughly analyzing, understanding the existing system.
- This process can be seen as reverse SDLC model, i.e. we try to get higher abstraction level by analyzing lower abstraction levels.
- An existing system is previously implemented design, about which we know nothing.
- Designers then do reverse engineering by looking at the code and try to get the design.
- With design in hand, they try to conclude the specifications.
- Thus, going in reverse from code to system specification.
- Reverse engineering can **extract design information from source code.**
- The abstraction level of a reverse engineering process refers to the sophistication of the design information that can be extracted from source code.
- Ideally, the abstraction level should be as high as possible.
- The reverse engineering process should be capable of;
    - deriving procedural design representations (a low-level abstraction)
    - program and data structure information (a somewhat higher level of abstraction)
    - object models, data flow models (a relatively high level of abstraction)
    - entity relationship models (a high level of abstraction).



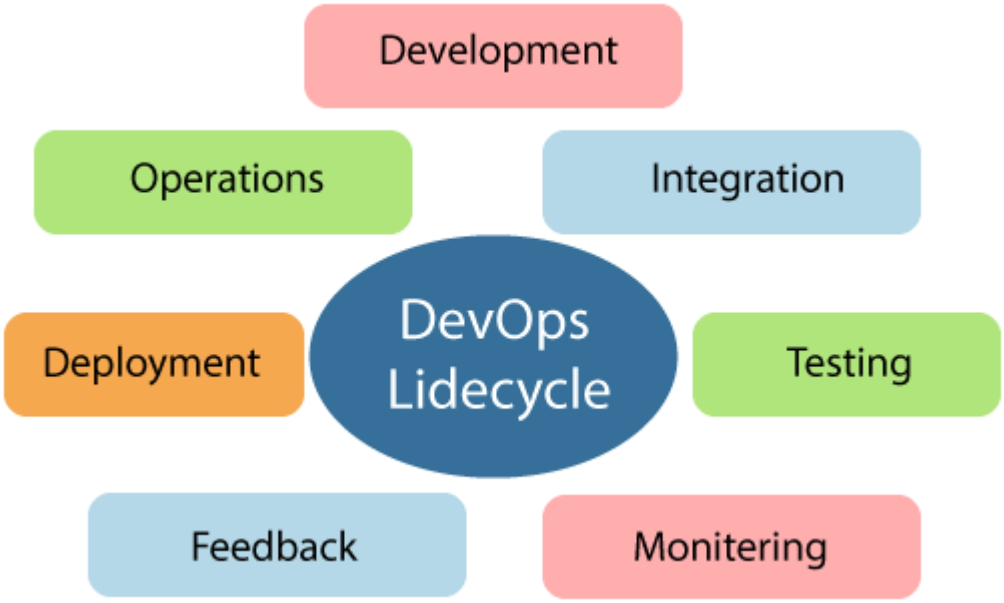
35) Difference between reverse engineering and forward engineering.

Forward Engineering	**Reverse Engineering**
In forward engineering, the application are developed with the given requirements.	In reverse engineering or backward engineering, the information are collected from the given application.
Forward Engineering is a high proficiency skill.	Reverse Engineering or backward engineering is a low proficiency skill.
Forward Engineering takes more time to develop an application.	While Reverse Engineering or backward engineering takes less time to develop an application.
The nature of forward engineering is Prescriptive.	The nature of reverse engineering or backward engineering is Adaptive.

Forward Engineering	Reverse Engineering
In forward engineering, production is started with given requirements.	In reverse engineering, production is started by taking the existing products.
The example of forward engineering is the construction of electronic kit, construction of DC MOTOR , etc.	An example of backward engineering is research on Instruments etc.
Forward engineering Starts with requirements analysis and design, then proceeds to implementation and testing.	Reverse engineering Starts with an existing software system and works backward to understand its structure, design, and requirements.
Forward engineering is used to create new software applications from scratch.	Reverse engineering is Used to modify and improve an existing software application.
Forward egineering is process of moving from a high-level abstraction to a detailed implementation.	Reverse engineering is a process of moving from a low-level implementation to a higher-level abstraction.
Requires a clear set of requirements and design specifications.	Requirements and design specifications may not be available, making it necessary to reconstruct them from the code itself.
Forward engineering is generally more time-consuming and expensive.	Reverse engineering is generally less time-consuming and less expensive.
The final product is completely new and independent of any existing software system.	The final product is typically a modified or improved version of an existing software system.
Involves a series of steps such as requirements gathering, design, implementation, testing, and deployment.	Involves steps such as code analysis, code understanding, design recovery, and documentation.
Forward engineering is commonly used in the initial stages of software development.	Reverse engineering is commonly used in the maintenance stage of the software development life cycle.

36) Explain 7 C’s of DevOps.

DevOps defines an agile relationship between operations and Development. It is a process that is practiced by the development team and operational engineers together from beginning to the final stage of the product.



Learning DevOps is not complete without understanding the DevOps lifecycle phases. The DevOps lifecycle includes seven phases as given below:

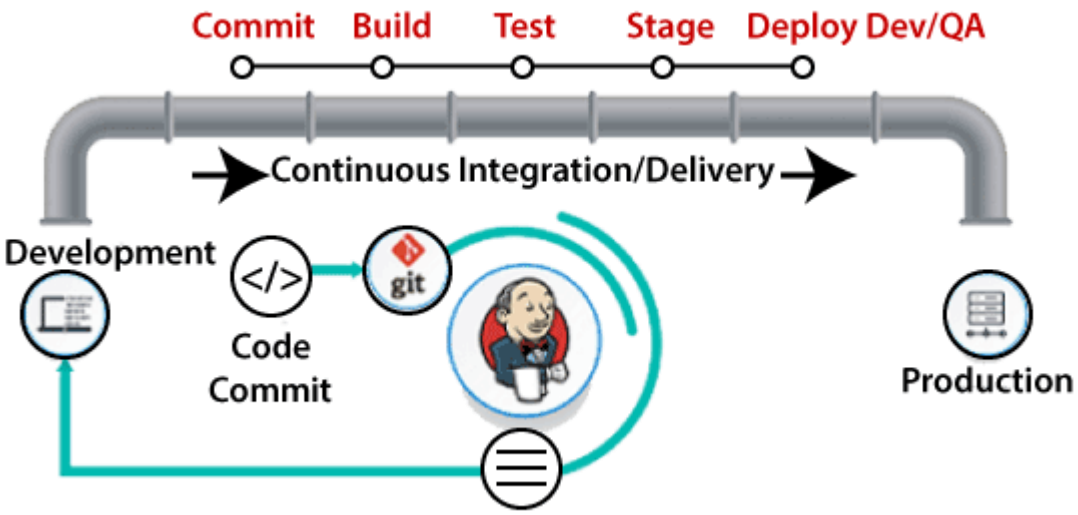
1) Continuous Development

This phase involves the planning and coding of the software. The vision of the project is decided during the planning phase. And the developers begin developing the code for the application. There are no DevOps tools that are required for planning, but there are several tools for maintaining the code.

2) Continuous Integration

This stage is the heart of the entire DevOps lifecycle. It is a software development practice in which the developers require to commit changes to the source code more frequently. This may be on a daily or weekly basis. Then every commit is built, and this allows early detection of problems if they are present. Building code is not only involved compilation, but it also includes **unit testing, integration testing, code review**, and **packaging**.

The code supporting new functionality is continuously integrated with the existing code. Therefore, there is continuous development of software. The updated code needs to be integrated continuously and smoothly with the systems to reflect changes to the end-users.



Jenkins is a popular tool used in this phase. Whenever there is a change in the Git repository, then Jenkins fetches the updated code and prepares a build of that code, which is an executable file in the form of war or jar. Then this build is forwarded to the test server or the production server.

3) Continuous Testing

This phase, where the developed software is continuously testing for bugs. For constant testing, automation testing tools such as **TestNG, JUnit, Selenium**, etc are used. These tools allow QAs to test multiple code-bases thoroughly in parallel to ensure that there is no flaw in the functionality. In this phase, **Docker** Containers can be used for simulating the test environment.

Selenium does the automation testing, and TestNG generates the reports. This entire testing phase can automate with the help of a Continuous Integration tool called **Jenkins**.

Automation testing saves a lot of time and effort for executing the tests instead of doing this manually. Apart from that, report generation is a big plus. The task of evaluating the test cases that failed in a test suite gets simpler. Also, we can schedule the execution of the test cases at predefined times. After testing, the code is continuously integrated with the existing code.

4) Continuous Monitoring

Monitoring is a phase that involves all the operational factors of the entire DevOps process, where important information about the use of the software is recorded and carefully processed to find out trends and identify problem areas. Usually, the monitoring is integrated within the operational capabilities of the software application.

It may occur in the form of documentation files or maybe produce large-scale data about the application parameters when it is in a continuous use position. The system errors such as server not reachable, low memory, etc are resolved in this phase. It maintains the security and availability of the service.

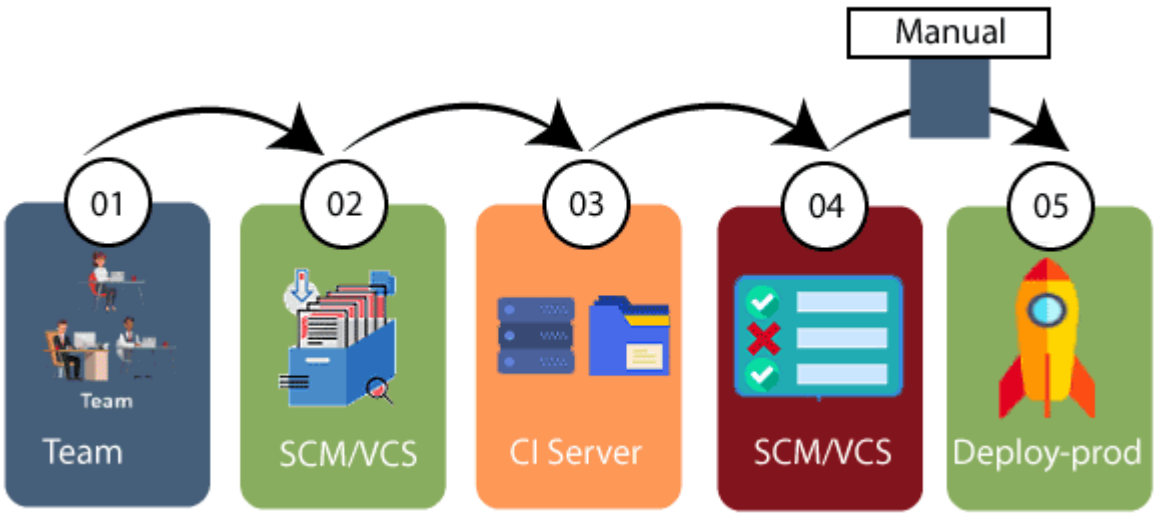
5) Continuous Feedback

The application development is consistently improved by analyzing the results from the operations of the software. This is carried out by placing the critical phase of constant feedback between the operations and the development of the next version of the current software application.

The continuity is the essential factor in the DevOps as it removes the unnecessary steps which are required to take a software application from development, using it to find out its issues and then producing a better version. It kills the efficiency that may be possible with the app and reduce the number of interested customers.

6) Continuous Deployment

In this phase, the code is deployed to the production servers. Also, it is essential to ensure that the code is correctly used on all the servers.



The new code is deployed continuously, and configuration management tools play an essential role in executing tasks frequently and quickly. Here are some popular tools which are used in this phase, such as **Chef, Puppet, Ansible**, and **SaltStack**.

Containerization tools are also playing an essential role in the deployment phase. **Vagrant** and **Docker** are popular tools that are used for this purpose. These tools help to produce consistency across development, staging, testing, and production environment. They also help in scaling up and scaling down instances softly.

Containerization tools help to maintain consistency across the environments where the application is tested, developed, and deployed. There is no chance of errors or failure in the production environment as they package and replicate the same dependencies and packages used in the testing, development, and staging environment. It makes the application easy to run on different computers.

7) Continuous Operations

All DevOps operations are based on the continuity with complete automation of the release process and allow the organization to accelerate the overall time to market continuingly.

It is clear from the discussion that continuity is the critical factor in the DevOps in removing steps that often distract the development, take it longer to detect issues and produce a better version of the product after several months. With DevOps, we can make any software product more efficient and increase the overall count of interested customers in your product.

37) Explain DevOps Principles.

DevOps Principles

The main principles of DevOps are Continuous delivery, automation, and fast reaction to the feedback.

1. **End to End Responsibility:** DevOps team need to provide performance support until they become the end of life. It enhances the responsibility and the quality of the products engineered.
2. **Continuous Improvement:** DevOps culture focuses on continuous improvement to minimize waste. It continuously speeds up the growth of products or services offered.
3. **Automate Everything:** Automation is an essential principle of the DevOps process. This is for software development and also for the entire infrastructure landscape.
4. **Custom Centric Action:** DevOps team must take customer-centric for that they should continuously invest in products and services.
5. **Monitor and test everything:** The DevOps team needs to have robust monitoring and testing procedures.
6. **Work as one team:** In the DevOps culture role of the designers, developers, and testers are already defined. All they needed to do is work as one team with complete collaboration.

These principles are achieved through several DevOps practices, which include frequent deployments, QA automation, continuous delivery, validating ideas as early as possible, and in-team collaboration.

38) List and explain the challenges in DevOps Implementation.

Here are the top challenges of implementing DevOps at your organization.

i. Change in Culture

The workplace culture undergoes the major amount of transformation while implementing DevOps. It is also one of the most difficult areas of transformation as it is a long term process which also requires a lot of patience and endurance. To make the process a bit easier, enterprises should try and maintain a positive as well as a transparent atmosphere in the workplace.

ii. Switching from Legacy Infrastructure to Microservices

In order to reduce stability issues, organisations now use infrastructure as code along with microservices for quicker development along with sharp innovations. Moreover, organisations need to update their hardware and software systems according to the latest trends on a regular basis, so that new systems can co-exist with the existing systems.

iii. Issues with the standards and metrics

Dev and Ops departments have different goals and working systems, hence they have different toolsets as well. It might become very tedious to sit together and integrate the tools. Under such circumstances, it is advisable that the teams agree upon a commonly decided metric system.

iv. Tool Turbulence

Switching to DevOps practices might make people dependent on the various tools that are available to solve even the smallest of their problems. Due to this, organisations might become addicted to those tools which provide with short-term benefits over the ones which provide with long-term benefits. Some of the tools are open-sourced or SaaS-based and can be easily adopted without any authorization. To make things easier, you can provide teams with a set of library tools from which they can opt for their preferred tools. This will also help the leaders stay up-to-date with the activities of the employees.

v. Resistance to Change

You might come across people in your company who might not be supportive of the legacy systems. They are the ones who have become comfortable with their way of working and are not willing to leave their comfort zones. Hence, it is very important that you don't bend down to such elements but instead bear with the discomfort of change.

vi. Challenges during the process

Adopting DevOps can prove to be challenging for workers who blindly follow guidelines and stay stuck to the rules, or for companies which follow specific guidelines for software development, as DevOps doesn't have any fixed framework stating procedures that employees can follow to reach their desired goals.

The teams can decide on their own course of action without any structural approach, giving them opportunities and more scope for innovation.

vii. Test Automation

Test Automation holds equal importance as CI/CD deployments. It has been commonly observed that companies tend to neglect test automation and focus more on CI/CD deployments. For DevOps to be a success, continuous testing acts as a key.

viii. Cost and Budget

It is very important to keep in mind that open source does not necessarily mean that it is free of cost. Moreover, factor in integration and operational complexity to your overall costs.